LogicGraph : Benchmarking Multi-Path Logical Reasoning via Neuro-Symbolic Generation and Verification

Yanrui Wu[1,2,3], **Lingling Zhang**[1,2,3*], **Xinyu Zhang**[1,2,3], **Jiayu Chang**[4], **Pengyu Li**[1,2,3], **Xu Jiang**[5], **Jingtao Hu**[1], **Jun Liu**[1,2,3]

[1]School of Computer Science and Technology, Xi'an Jiaotong University, China
[2]National Engineering Research Center for Visual Information and Applications
[3] Shaanxi Province Key Laboratory of Big Data Knowledge Engineering
[4]Department of Electrical Engineering, Stanford University
[5]School of Computer Science and Technology, Tiangong University
yanrui.wu@stu.xjtu.edu.cn, zhanglling@xjtu.edu.cn

Abstract

Evaluations of large language models (LLMs) primarily emphasize convergent logical reasoning, where success is defined by producing a single correct proof. However, many real-world reasoning problems admit multiple valid derivations, requiring models to explore diverse logical paths rather than committing to one route. To address this limitation, we introduce **LogicGraph**, the first benchmark aimed to systematically evaluate multi-path logical reasoning, constructed via a neuro-symbolic framework that leverages backward logic generation and semantic instantiation. This pipeline yields solver-verified reasoning problems formalized by high-depth multi-path reasoning and inherent logical distractions, where each instance is associated with an exhaustive set of minimal proofs. We further propose a reference-free evaluation framework to rigorously assess model performance in both convergent and divergent regimes. Experiments on state-of-the-art language models reveal a common limitation: models tend to commit early to a single route and fail to explore alternatives, and the coverage gap grows substantially with reasoning depth. LogicGraph exposes this divergence gap and provides actionable insights to motivate future improvements. Our code and data will be published at https://github.com/kkkkarry/LogicGraph/.

1 Introduction

Logical reasoning is central to general intelligence, enabling systems to derive valid conclusions from given premises (Kaufman et al., 2011). However, most existing benchmarks for evaluating LLM reasoning (Han et al., 2024; Xu et al., 2025; Huang et al., 2025; Zhang et al., 2025) primarily emphasize *convergent* thinking—assessing whether a model can reach a correct final conclusion for a given problem. This emphasis leaves underexplored what Guilford terms *divergent* thinking (Guilford, 1967): the ability to actively generate multiple plausible alternatives for the same conclusion in a problem.

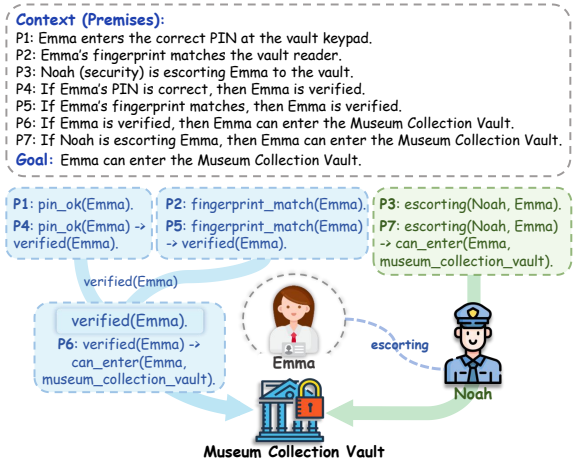


Figure 1: Illustration of the multi-path reasoning challenge. In the real world, the same conclusion may be entailed via multiple derivation paths.

In real-world scenarios, reasoning is rarely linear; agents may reach the same conclusion through multiple valid lines of argument and often benefit from exploring alternatives (Yao et al., 2023; Besta et al., 2024; Zhang et al., 2026). Consider the access-control scenario in Figure 1. The conclusion "Emma can enter the Vault" can be entailed via distinct mechanisms, e.g., through PIN verification (P1, P4, P6), biometrics (P2, P5, P6), or a security escort (P3, P7), illustrating that multiple derivation paths may support the same claim. To bridge this gap between real-world reasoning demands and current benchmark objectives, we formulate the multi-path logical reasoning task: given premises and a conclusion, the goal is to enumerate the distinct logically valid derivation paths that entail the conclusion, rather than merely outputting

*Corresponding author

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 15931–15959
October 24-29, 2026 ©2026 Association for Computational Linguistics

Benchmark	Symbolic Notation	Distraction	Depth (avg.)	Paths (range)	Reuse Ratio (range)	Task Type	Stepwise Evaluation
ProofWriter	✗	✗	2.2	1	1.0	Binary	✗
FOLIO	✓	✗	3.4	1	1.0	Binary	✗
ProntoQA	✓	✗	–	–	–	Binary	✗
RuleTaker	✗	✗	2.4	1	1.0	Binary	✗
LogicBench	✗	✗	–	–	–	Binary/MCQA	✗
LogicNLI	✗	✗	–	–	–	Ternary	✗
Multi-LogiEval	✗	✗	2.51	1	1.0	Binary	✗
ProverQA	✓	✓	4.73	1	1.0	Ternary	✗
LogicGraph	✓	✓	**6.01**	**2 ∼19**	**1.0 ∼1.9**	**Proof Gen.**	✓

Table 1: Comparison of LogicGraph (ours) with existing logical reasoning datasets across key characteristics. Distraction refers to the simultaneous occurrence of logical and semantic distraction.

a binary label.

However, establishing a benchmark for multi-path reasoning faces three fundamental challenges: *(i) Scalable Construction* of exhaustive ground-truth derivation paths; *(ii) Reliable Evaluation* of open-ended generation; and *(iii) Cognitive Assessment* of exploration versus mere correctness. To address these challenges, we introduce LogicGraph, the first benchmark and evaluation framework designed for multi-path logical reasoning.

Construction via Goal-Driven Logic DAGs. Manually annotating every valid reasoning path in complex text is prohibitively error-prone. To guarantee exhaustive ground truth, we employ a goal-driven backward construction paradigm (Al-Ajlan, 2015). We synthesize premises from the conclusion upwards using fundamental argument forms, creating a symbolic logic Directed Acyclic Graph (DAG) that is subsequently verbalized. This DAG-based construction yields three characteristics of LogicGraph: high-depth multi-path reasoning (with 2–19 valid paths per query and an average depth of 6.01), inference node reuse, and inherent logical distractions. See Table 1 for more details.

Neuro-Symbolic Evaluation. Evaluating divergent outputs is difficult because traditional string matching is brittle and LLM-as-a-Judge approaches suffer from hallucination (Wang et al., 2025; Szymanski et al., 2025). We propose a *reference-free neuro-symbolic* evaluator that translates generated natural language steps into formal logic and verifies them using a symbolic solver (Prover9 (McCune, 2005)). This pipeline rigorously verifies both local step validity and global proof reachability, achieving high agreement with human experts (98.80% Step Accuracy and 95.22% Proof Accuracy), thus ensuring a reliable metric for open-ended reasoning.

Cognitive Assessment & Insights. Moving beyond aggregate accuracy metrics, we conduct a dual-faceted analysis of *Convergent* and *Divergent* thinking capabilities. We evaluate a broad range of state-of-the-art LLMs on LogicGraph, encompassing both general-purpose models, such as GPT-5.5 (OpenAI, 2026), and reasoning-oriented models, such as Gemini-3-Pro (Deepmind, 2025c). Our results reveal a significant performance gap: while current models demonstrate competence in *Convergent* metrics, their *Divergent* thinking capabilities remain significantly constrained, characterized by a sharp decline in solution coverage as logical complexity increases. Although reasoning-oriented models generally outperform general models, the best model fails to achieve reliable exploratory reasoning. Furthermore, a fine-grained error analysis indicates that failures are predominantly "result-oriented"; models frequently hallucinate intermediate lemmas to artificially force connections to target conclusions. These insights underscore the substantial gap between current model capabilities and the requirements for divergent thinking.

2 Related Work

2.1 Logical Reasoning Datasets for LLMs

Logical reasoning benchmarks for LLMs have evolved along three lines. Exam-derived datasets such as LogiQA (Liu et al., 2020), AR-LSAT (Zhong et al., 2021), and ReClor (Yu et al., 2020) are derived from standardized tests but offer limited control over reasoning depth. Synthetic benchmarks led by RuleTaker (Clark et al., 2020) and ProofWriter (Tafjord et al., 2021) were intro-

duced, later extended with explicit logical forms and harder multi-step chains (Tian et al., 2021; Saparov and He, 2022; Morishita et al., 2023; Han et al., 2024; Patel et al., 2024; Parmar et al., 2024; Chen et al., 2025). SAT-style benchmarks (Wei et al., 2025; Hazra et al., 2025) likewise terminate once a satisfying assignment is found. (Bok and Chua, 2025) studies divergent thinking in finance, but lacks enumerable ground truth over logical alternatives. Overall, most existing benchmarks emphasize answer correctness while underexploring the multi-path nature of derivation.

2.2 Symbolic Prover Augmented LLMs

Symbolic provers (e.g., Z3 (De Moura and Bjørner, 2008) and proof assistants such as Lean (De Moura et al., 2015)) offer trusted verification of formal statements, motivating neuro-symbolic pipelines that couple LLMs with prover checking. Prior work typically uses LLMs to translate natural language into executable logical forms and delegate solving/verification to external engines (Pan et al., 2023; Deng et al., 2024; Shen et al., 2025; Xu et al., 2024a; Wang et al., 2024a). Provers are exploited for data generation and filtering to ensure logical consistency of synthesized corpora (Qi et al., 2025).

3 Task Formulation

We formulate the reasoning task as finding all valid justifications for a goal hypothesis $\mathcal{G}$ given a set of premises $\mathcal{P} = p_1, p_2, \ldots, p_n$. We refer to each distinct justification as a *derivation path*.

Formally, a derivation path is defined as a *minimal support* set. A set $S \subseteq \mathcal{P}$ is a minimal support for $\mathcal{G}$, denoted by $\mathrm{MinSup}(S, \mathcal{G})$, if and only if (i) S entails $\mathcal{G}$, (ii) no proper subset of S entails $\mathcal{G}$:

$$S \vdash \mathcal{G} \quad \text{and} \quad \forall S' \subsetneq S : \; S' \nvdash \mathcal{G}. \qquad (1)$$

The task is to enumerate all such sets S that satisfy $\mathrm{MinSup}(S, \mathcal{G})$. As shown in Figure 1, $S_{\text{escort}} = \{P3, P7\}$ is a minimal support for $\mathcal{G}$: removing either $P3$ or $P7$ breaks entailment. Other minimal supports correspond to alternative routes (e.g., PIN- or fingerprint-based).

4 Automatic Dataset Generation Pipeline

To ensure both logical rigor and linguistic diversity, we design a three-stage automated pipeline. Stage 1 constructs a symbolic directed acyclic graph to serve as a verifiable reasoning skeleton; Stage 2 translates this symbolic structure into coherent natural language contexts to simulate real-world complexity; Stage 3 employs a symbolic solver to verify the validity and consistency of the generated data, filtering out invalid samples.

4.1 Symbolic Logic DAG Generation

To mitigate logical inconsistencies in directly generated reasoning texts, we first construct a symbolic directed acyclic graph (Logic DAG) that specifies the ground-truth reasoning structure independent of natural language. As shown in Figure 2(a), we build this graph via goal-driven backward construction from a target conclusion, which will later be used for semantic instantiation.

Graph Primitives: Inference Node & Family. A reasoning path is a sequence of *inference nodes*, each being a rule instance $(\Gamma \Rightarrow \phi)$ that derives conclusion ϕ from a *local* premise set Γ (here Γ is distinct from the global given premise pool $\mathcal{P}$ in Section 3). We further group paths into *families* if they share the same set of inference nodes.

Goal-Driven Construction. We begin by sampling a conclusion node. Then, for each current node N, we (i) sample a form from a predefined set of fundamental forms (Johnson, 1999) (e.g., Modus Ponens; formal definitions in Appendix A.1), and (ii) generate its parent premises. For example, to derive Q via Modus Ponens, we create node $(P \to Q)$ and node P, and recursively expand the newly created premise nodes.

Multi-path Generation. To generate multiple reasoning paths, we first build a reasoning chain to a sampled depth, then pick an intermediate conclusion on the chain and expand it upward again using the same goal-driven procedure. Repeating this step produces a Logic DAG in which the goal is supported by multiple distinct premise sets. To ensure the ground-truth solution set is exhaustive, we assign a fresh atomic identifier to each newly introduced premise unless it is explicitly shared through an existing node, thereby avoiding unintended premise sharing and implicit extra paths. Appendix A.4.1 walks through this on an example and shows why exhaustiveness holds.

4.2 Semantic Instantiation

Although the Logic DAG ensures validity, it is not semantically rich enough for LLM evaluation.

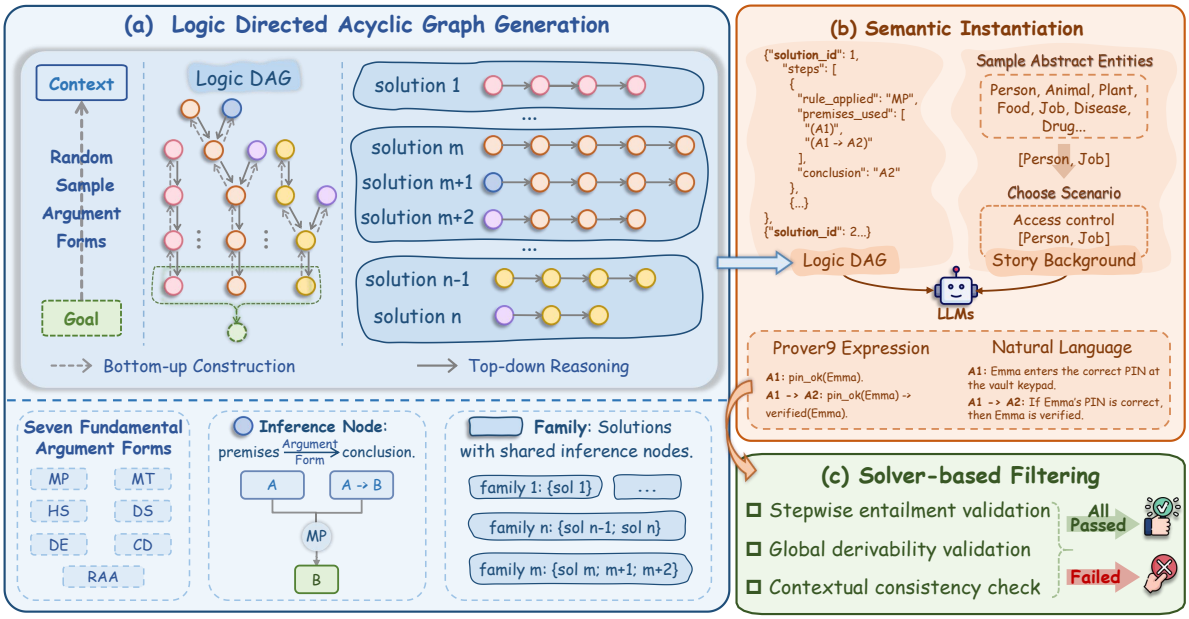


Figure 2: LogicGraph generation pipeline. (a) Logic DAG generation (Section 4.1) builds a goal-to-premise directed acyclic graph by sampling argument forms, yielding multiple valid reasoning paths; paths that share intermediate inference nodes are grouped into *families*. (b) Semantic instantiation (Section 4.2) translates the DAG into Prover9 formulas and renders the corresponding steps into natural language by instantiating abstract entities and scenario context with LLMs. (c) Solver-based Filtering (Section 4.3) uses Prover9 to validate each instance.

We thus transform each Logic DAG into a natural-language test case through an intermediate Prover9 form, which also enables downstream verification. Following prior work, we define 32 abstract entity types (e.g., *Person*, *Job*) (Rosch and Mervis, 1975; Wang et al., 2024b). For each DAG, we sample a domain background, use DeepSeek-V3.2-Exp to instantiate abstract symbols into domain-specific Prover9 predicates (e.g., convert A1 to pin_ok(Emma)), and then verbalize the instantiated formulas into a coherent narrative while preserving all logical relations and avoiding unsupported facts.

4.3 Solver-based Filtering

Multi-path reasoning requires verification to ensure both edge-level (stepwise) soundness and graph-level consistency. Based on the Prover9 expressions produced in Sec. 4.2, we validate every sample with Prover9 and discard any sample that fails one of the following checks: (i) Stepwise Entailment. For each inference step (i.e., each edge in the Logic DAG), Prover9 verifies that the corresponding premises entail the derived statement, ensuring that every deduction is logically valid. (ii) Global Derivability. Prover9 confirms that all intermediate statements are derivable from the given premises and earlier derivations, and that the target goal is derivable from the full set of derivations, ensuring the graph is globally connected. (iii) Contextual Consistency. The union of all premises across multiple paths must be satisfiable (e.g., avoiding simultaneous $Fact(X)$ and $\neg Fact(X)$), ensuring no contradictions exist within the global context.

4.4 Dataset Characteristics

Based on the automatic generation pipeline described above, LogicGraph is scalable and robust against contamination. Using this dynamic pipeline, we curated a statistically robust benchmark of 900 instances via stratified sampling. We further divide the dataset into three difficulty tiers according to the number of valid derivation paths (n): *Small* ($2 \leq n \leq 4$), *Medium* ($5 \leq n \leq 7$), and *Large* ($n \geq 8$), with 300 instances per tier.

As illustrated in Table 1, LogicGraph exhibits three structural properties *induced by graph-based generation* that distinguish it from prior reasoning benchmarks: **(1) Multi-path High-depth.** Unlike predominantly shallow and single-path datasets, each LogicGraph query admits 2–19 *valid* proof paths with an average depth of 6.01. This requires not only long-chain deduction but also exploration over multiple proof trajectories. **(2) Inference Node Reuse.** Unlike chain-style proofs where inter-

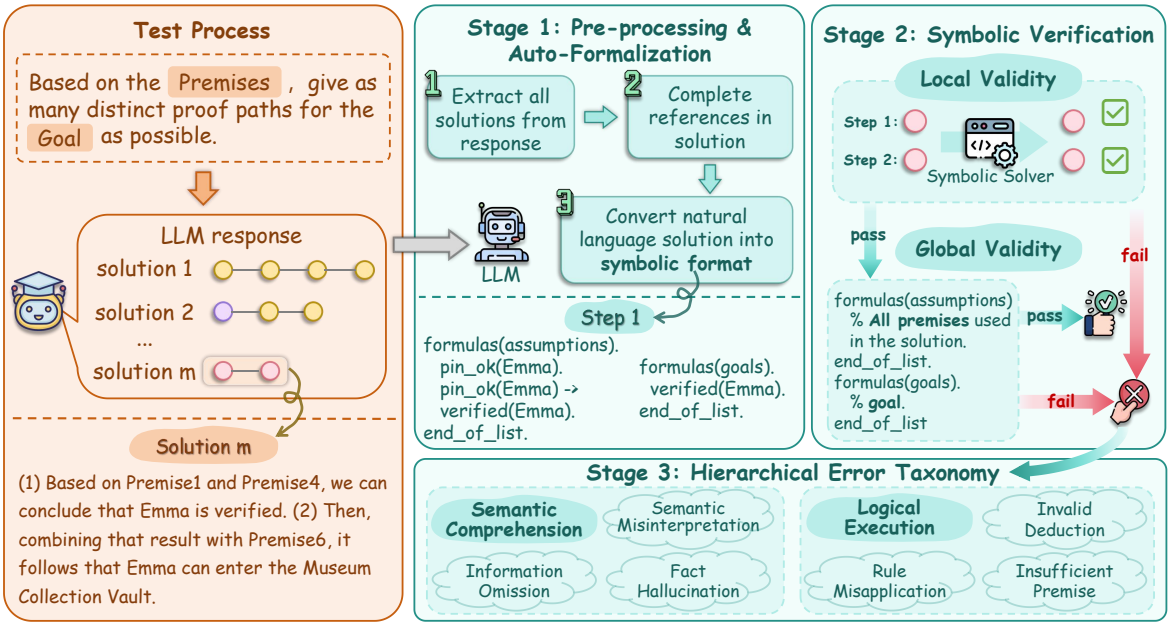


Figure 3: Three-stage evaluation pipeline for LLM-generated multi-path proofs: (1) pre-processing and auto-formalization extract candidate solutions, resolve references, and translate natural-language steps into a symbolic representation (Prover9-style); (2) a solver is then used to validate local (stepwise) and global validity; (3) failures are annotated along two independent, non-exclusive error axes: semantic comprehension and logical execution.

mediate conclusions are path-specific, LogicGraph allows intermediate inference nodes to be shared across branches (Reuse Ratio: 1.0–1.9). Consequently, models must identify and reuse shared subproofs rather than solving each path independently. **(3) Inherent Logical Distractions.** Beyond static semantic noise or dead-end facts, LogicGraph introduces structural distractions from the DAG itself: a premise can be crucial for one valid path yet distracting for another. Models must selectively activate the premises supporting a target trajectory from logically valid but competing evidence.

5 Neuro-Symbolic Evaluation Framework

The rapid advancement of LLMs in complex reasoning necessitates a more rigorous evaluation paradigm. Current evaluation methods often rely on LLM-as-a-Judge. Yet this purely neural paradigm inherits the same weaknesses it aims to diagnose—hallucinations, weak formal grounding, and systematic biases in self-correction—making it easy to miss subtle logical fallacies (Szymanski et al., 2025; Wang et al., 2025; Feng et al., 2025). Conversely, traditional symbolic solvers provide deterministic verification but remain too rigid to handle the informal reasoning traces.

To overcome these limitations, we integrate an LLM as a flexible formal translator and a symbolic solver as a rigorous verifier (Bhuyan et al., 2024). In the following sections, we detail the evaluation pipeline, define the metrics, and present a meta-evaluation to validate the framework's reliability.

5.1 The Evaluation Pipeline

As illustrated in Figure 3, our framework evaluates the logical validity of the entire reasoning process via a three-stage pipeline.

Stage 1: Pre-processing & Auto-Formalization. To support symbolic verification, we disentangle distinct derivation paths from the model output and reconstruct implicit dependencies (e.g., "From the previous step…") into an explicit chain. An LLM then translates each NL step into Prover9 syntax. To ensure fidelity, we use in-context examples aligning Prover9 expressions from the data synthesis stage with their corresponding NL descriptions.

Stage 2: Symbolic Verification. We employ Prover9 to verify the reasoning chain at two levels: (i) *Local Validity* checks whether each step S_t logically follows from its cited premises P_t (i.e., $P_t \vdash S_t$), (ii) *Global Validity* ensures whether the final conclusion G can be derived solely from the subset of premises explicitly used in the solution.

Dimension	Error Type	Definition
Semantic Comprehension	Semantic Misinterpretation	*Incorrectly understood the meaning of facts or rules.*
	Information Omission	*Ignored necessary premises or constraints explicitly stated in the context.*
	Fact Hallucination	*Used external or fabricated premises not present in the input.*
Logical Execution	Invalid Deduction	*The conclusion does not logically follow from the cited premises.*
	Rule Misapplication	*Applied a rule that doesn't match the current premises.*
	Insufficient Premise	*Cited relevant premises but incomplete for deriving the conclusion.*

Table 2: The proposed hierarchical error taxonomy. The taxonomy distinguishes between failures in processing input information (Comprehension) and failures in the reasoning process (Execution).

Together, these checks ensure grounded proofs and flag hallucinated steps.

Stage 3: Hierarchical Error Taxonomy. To pinpoint where and why models fail, we propose a two-dimensional error taxonomy (Table 2):

Semantic Comprehension. Faithfulness to the provided context, for example, when the model hallucinates a relationship not stated in the premises.

Logical Execution. Validity of derivation steps, such as applying an incorrect rule or producing a non-sequitur conclusion.

5.2 Cognitive Evaluation Metrics

To capture the multifaceted nature of LLM reasoning, we adopt a two-axis evaluation protocol inspired by Guilford's Structure of Intellect (Guilford, 1967), evaluating convergent and divergent thinking beyond outcome accuracy.

Convergent Thinking assesses the model's ability to derive a single correct conclusion through valid logical steps. We quantify this via three primary dimensions: (i) *Success Rate*, the proportion of test cases with at least one valid proof path; (ii) *Precision*, the ratio of valid solutions to total generations, reflecting hallucination resistance; and (iii) *Shortest Path Finding Rate*, the percentage of solutions matching the minimum ground-truth step count, which measures reasoning conciseness.

Divergent Thinking evaluates creativity and flexibility in discovering multiple distinct paths. This is measured by: (i) *Diversity (Solution Recall)*, defined as $R_{\text{sol}} = |\mathcal{S}_{Model} \cap \mathcal{S}_{GT}| / |\mathcal{S}_{GT}|$, which quantifies the coverage of the solution space; (ii) *Versatility (Family Recall)*, which reflects the agility to switch between distinct reasoning strategies; and (iii) *Originality*, which highlights the ability to identify rare paths by calculating the inverse frequency of a solution's discovery across all models.

5.3 Evaluator Reliability Validation

To validate our framework, we measure agreement with external human judgments on proof validity and compare against reference-based LLM-as-a-Judge baselines. As shown in Table 3, our reference-free neuro-symbolic evaluator (with Prover9) is robust across backbone LLMs: both the Gemini-2.5-Flash and DeepSeek-V3.2-Exp versions outperform reference-based LLM-as-a-Judge baselines, with DeepSeek-V3.2-Exp achieving the highest agreement. This suggests symbolic verification reduces false positives from fluent-but-invalid traces. A blind external audit further confirms that the human judgments are reproducible, while translator-backbone swaps preserve model rankings; full results are reported in Appendix D.2.

Evaluator	Reference	Acc(S)	Acc(P)
LLM-as-a-Judge Baselines			
DeepSeek-V3.2-Exp	Required	77.25	71.46
DeepSeek-V3.2-T*	Required	87.19	83.59
Gemini-3-Pro	Required	86.11	83.91
Neuro-Symbolic			
Ours (Gemini)	None	97.57	94.85
Ours (DeepSeek)	**None**	**98.80**	**95.22**

Table 3: Meta-evaluation results comparing agreement with human annotators. Acc(S) and Acc(P) denote per-step and overall proof validity, respectively.

6 Experiments

6.1 Experimental Setting

Baselines. We evaluate 23 SOTA LLMs on LogicGraph, including both proprietary APIs and open-weight releases. The *proprietary* models include GPT-5.1 (OpenAI, 2025a), GPT-5.5 (OpenAI, 2026), o3 and o4-mini (OpenAI, 2025c), Claude-Sonnet-4.5 (Anthropic, 2025), and Gemini-2.5/3/3.5 (Deepmind, 2025a,b,c, 2026). The *open-weight* models include GLM-4.6 (ZhipuAI, 2025), Kimi-K2 (Bai et al., 2025), GPT-

Model	Convergent (↑) SR Avg; S/M/L	Convergent (↑) Prec. Avg; S/M/L	Convergent (↑) SPF Avg; S/M/L	Divergent (↑) Div. Avg; S/M/L	Divergent (↑) Vers. Avg; S/M/L	Divergent (↑) Orig. Avg; S/M/L	Token Eff. (Overall)↓
General Models							
GLM-4.6	**21.11** 19.67/21.00/22.67	**8.47** 11.20/7.62/6.58	**14.00** 15.00/13.00/14.00	**6.19** 9.83/5.30/3.45	**9.66** 13.50/8.35/7.14	**9.95** 8.70/10.09/11.07	7274.96
GPT-OSS-20B	**8.00** 9.76/5.78/8.47	**8.34** 10.14/6.08/8.79	**4.51** 7.07/4.08/2.37	**1.90** 3.76/1.03/0.92	**3.51** 6.71/1.77/2.06	**0.72** 0.75/0.51/0.91	50170.27
GPT-OSS-120B	**12.22** 13.67/11.33/11.67	**9.97** 12.07/8.06/9.79	**7.89** 9.67/8.67/5.33	**3.09** 5.75/2.12/1.41	**5.50** 9.75/3.76/2.99	**8.47** 9.04/8.08/8.29	27120.45
Qwen3-235B-A22B	**29.56** 30.67/27.67/30.33	**13.89** 17.47/12.69/11.52	**17.33** 20.67/17.00/14.33	**8.23** 13.28/6.60/4.80	**14.65** 21.67/12.14/10.14	**11.33** 9.62/11.16/13.22	7401.54
Claude-Sonnet-4.5	**53.11** 52.00/51.67/55.67	**36.05** 39.60/34.09/34.47	**29.22** 31.33/30.67/25.67	**12.88** 19.94/10.97/7.74	**25.42** 37.00/21.96/17.31	**11.79** 10.82/12.02/12.54	3431.10
DeepSeek-V3.2-Exp	**61.00** 57.33/61.67/64.00	**52.04** 49.34/49.90/56.89	**34.44** 33.67/35.33/34.33	**16.54** 25.39/14.51/9.71	**31.38** 43.67/28.64/21.84	**12.51** 11.39/12.50/13.64	3397.51
GPT-5.5	**78.89** 75.33/81.33/80.00	**78.56** 74.81/79.50/81.37	**66.67** 64.67/69.67/65.67	**49.01** 54.97/51.86/40.20	**62.61** 65.92/64.63/57.29	**20.10** 14.78/20.24/25.28	1789.08
Reasoning-Oriented Models							
o3	**58.33** 54.33/55.00/65.67	**50.79** 47.77/47.39/57.22	**39.44** 43.00/38.33/37.00	**17.94** 28.28/14.70/10.84	**30.02** 41.33/25.71/23.01	**12.10** 10.62/12.17/13.50	9446.91
o4-mini	**57.78** 56.67/57.33/59.33	**51.38** 48.99/49.40/55.76	**33.67** 38.00/32.33/30.67	**16.17** 25.14/14.04/9.34	**29.20** 40.42/27.21/19.97	**11.51** 10.72/11.79/12.01	6903.90
QwQ-32B	**50.89** 48.00/49.00/55.67	**25.91** 27.66/24.25/25.81	**35.44** 34.33/37.33/34.67	**16.20** 23.81/14.48/10.32	**27.38** 34.97/25.13/22.04	**11.53** 10.15/11.83/12.62	12228.78
Qwen3-235B-A22B-T*	**91.22** 85.00/93.33/95.33	**76.89** 73.08/78.49/79.11	**70.22** 68.00/75.33/67.33	**45.83** 59.42/45.42/32.64	**69.12** 76.67/69.28/61.42	**17.39** 14.94/17.62/19.61	5940.66
Kimi-K2-T*	**72.22** 69.67/74.67/72.33	**48.20** 46.51/50.67/47.42	**51.44** 56.00/54.33/44.00	**29.04** 42.06/27.68/17.37	**44.94** 58.08/44.52/32.22	**14.30** 12.71/14.34/15.86	13764.23
Claude-Sonnet-4.5-T*	**61.67** 61.67/59.67/63.67	**47.82** 50.58/46.05/46.82	**34.22** 40.00/34.00/28.67	**15.66** 25.47/13.07/8.43	**30.15** 45.31/26.23/18.92	**11.97** 11.06/11.74/13.10	3476.57
DeepSeek-V3.2-Exp-T*	**88.00** 82.00/89.33/92.67	**76.93** 71.02/77.83/81.93	**66.33** 67.33/69.67/62.00	**39.75** 53.81/39.22/26.21	**61.00** 70.67/61.32/51.00	**15.51** 13.65/15.72/17.17	5584.53
Gemini-2.5-Pro	**87.78** 82.67/88.67/92.00	**73.38** 72.20/72.90/75.03	**72.78** 72.67/75.67/70.00	**51.07** 60.56/53.11/39.53	**69.13** 74.50/69.83/63.07	**20.29** 15.66/20.02/25.20	4414.16
Gemini-3-Pro-Preview	**96.11** 93.00/98.00/97.33	**90.10** 89.11/91.89/89.31	**80.33** 80.00/82.67/78.33	**59.60** 70.03/60.30/48.47	**79.44** 83.03/79.99/75.30	**21.81** 17.39/21.69/26.35	1302.15
Gemini-3.5-Flash	**91.00** 88.00/92.67/92.33	**92.10** 89.26/93.18/93.87	**80.11** 76.33/84.33/79.67	**64.78** 68.69/67.32/58.32	**76.01** 77.47/77.66/72.91	**26.06** 19.02/25.48/33.67	4362.72

Table 4: Comprehensive model comparison on the LogicGraph, covering convergent metrics (Success Rate/Precision/Short Path Finding Rate), divergent metrics (Diversity/Versatility/Originality), and overall TokenEfficiency (↓) across Small/Medium/Large. In each cell, **Avg** (over Small/Medium/Large) is bolded; the second line reports Small/Medium/Large in order. -T* denotes Thinking. Best score in each metric is highlighted in blue.

OSS-20B/120B (OpenAI, 2025b), Llama-3.1-8B-Instruct (Meta, 2024), Qwen2.5-7B-Instruct (Yang et al., 2024), Qwen3-8B/14B/235B-A22B (Yang et al., 2025), QwQ-32B (Team, 2025), and DeepSeek-V3.2-Exp (DeepSeek-AI, 2025); we additionally evaluate the official Thinking variants of Qwen3-235B-A22B, Claude-Sonnet-4.5 and DeepSeek-V3.2-Exp. For all models we use a unified prompting protocol that encourages as many independent and verifiable solution paths as possible under a fixed structured answer template; prompt templates are in Appendix F.

Evaluation Flow. Following the evaluation results in Sec. 5.1, we select DeepSeek-V3.2-Exp as the default backbone model for our experiments; see Appendix G for the detailed prompt.

6.2 Main Results

Table 4 compares General and Reasoning-Oriented models on LogicGraph. For readability we keep only the strongest model from each series and a few informative baselines; dominated variants are deferred to Table 11 and do not affect the conclusions below. We highlight three key observations:

Reasoning helps, and the frontier achieves it without extra cost. Overall, reasoning-oriented models outperform general models on convergent metrics, indicating that stronger inference is essential for logic tasks. Notably, Gemini-3-Pro and Gemini-3.5-Flash establish a new state-of-the-art on all metrics, while also exhibiting the highest token efficiency, suggesting that improved reasoning reduces wasted generation on incorrect branches and reaches valid reasoning paths more directly.

Model	Attempt Cov. S/M/L	Validity-of-Attempt S/M/L
Gemini-3.5-Flash	75.4/72.0/60.0	91.7/93.5/94.0
Gemini-3-Pro	77.4/65.5/52.2	89.0/91.8/89.4
DeepSeek-V3.2-T*	72.8/49.9/30.4	71.6/77.9/81.8

Table 5: Exploration–execution decomposition (%) across Small/Medium/Large path-count tiers.

Tier	n	Default	Target-k	Δ
Small	10	63.3	65.0	+1.7
Medium	10	47.1	29.0	−18.1
Large	10	33.0	10.7	−22.3
Overall	30	47.8	34.9	−12.9

Table 6: Target-count prompt ablation on a balanced 30-instance subset. Default and Target-k report Diversity (%); Δ is in percentage points.

Top models remain concentrated on a narrow subset of valid reasoning paths. Despite near-saturated Success Rates for top models, divergent metrics remain markedly lower, revealing a gap between finding one valid reasoning path and enumerating many. This indicates that current models tend to concentrate on a limited subset of high-probability paths rather than systematically exploring diverse alternatives.

Multiplicity: Models can switch strategies, but cannot enumerate many valid paths. As the number of valid reasoning paths increases from Small to Large, both Versatility and Diversity decline across models. However, Versatility is typically more robust than Diversity, indicating that models can still reach multiple high-level strategies but increasingly fail to exhaustively list the many concrete variants implied by each strategy.

6.3 Disentangling the Divergence Gap

Exploration vs. Execution. We separate generation scale from execution quality using *Attempt Coverage* (attempted/ground-truth paths) and *Validity-of-Attempt* (valid/attempted paths), both computed before support-level deduplication. As shown in Table 5, Attempt Coverage drops sharply with the number of valid paths, whereas Validity-of-Attempt remains stable or improves. Models thus remain reliable on attempted paths but fail to scale exploration with the solution space, indicating early satisficing rather than deteriorating per-path execution.

Target-Count Prompting. On a balanced 30-instance pilot with Gemini-2.5-Flash, revealing the exact number of proofs changes Diversity by $+1.7$, -18.1, and -22.3 points on Small, Medium, and Large instances, respectively (Table 6). Explicit target counts therefore do not recover the missing alternatives, suggesting that under-exploration is not simply caused by uncertainty about how many proofs to produce. This single-model pilot is indicative rather than conclusive.

6.4 Depth-Induced Performance Degradation

As shown in Figure 4, both Success Rate and Precision decline steadily with increasing reasoning depth, indicating that errors accumulate across multi-hop chains. Reasoning models remain more robust in the mid-to-high step ranges, while general models degrade more sharply, suggesting reasoning optimization strengthens cross-hop consistency and error tolerance. Beyond correctness, Diversity and Versatility drop earlier and faster, implying that deeper puzzles drive models toward narrower reasoning patterns and explore fewer alternatives. Controlled analyses further show that this degradation persists after accounting for the numbers of valid paths, premises, and reasoning families (Appendix D.3).

6.5 Limits of Search-based Reasoning

We test whether search-based strategies (ToT (Yao et al., 2023), GoT (Besta et al., 2024)) close the divergence gap. As shown in Table 17, ToT uniformly hurts performance, while GoT helps weak backbones but degrades strong ones, indicating that graph-structured decomposition compensates for limited reasoning capacity rather than amplifying strong reasoning. See Appendix D.4 for details.

6.6 Analysis of Error Categories

Figure 5 presents the error types distribution across representative models (Appendix B provides results for all evaluated models), while most models maintain relatively low error rates in semantic comprehension, they exhibit a pronounced spike in logical execution errors, particularly invalid deduction and insufficient premise. This suggests that in proof tasks, LLMs may generate result-oriented fabrications. When a valid logical chain from premises to conclusion is unavailable, they tend to invent intermediate lemmas or make unjustified leaps to produce a superficially complete proof.

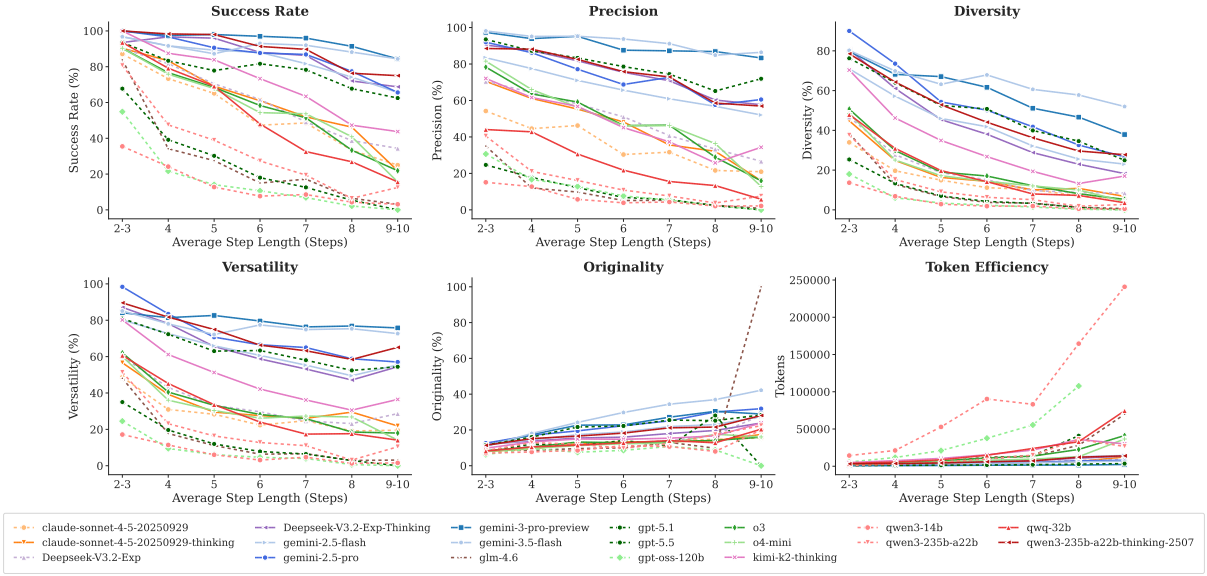


Figure 4: Performance dynamics of LLMs across varying reasoning depths. Solid lines represent Reasoning-oriented LLMs; dashed lines represent General-purpose LLMs.

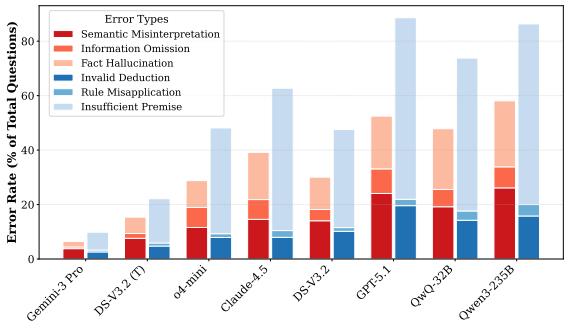


Figure 5: Comparative Analysis of Error Type: Semantic Comprehension vs. Logical Execution.

6.7 Case Study

Figure 11 shows why we need Divergent Metrics beyond standard scores. Further analysis uncovers hidden reasoning flaws even when answers are correct; see Appendix C.1 for more details.

7 Conclusion

We develop a neuro-symbolic generation and solver-verification pipeline for constructing high-depth multi-path logical reasoning problems. Based on it, we build LogicGraph, the first benchmark where each instance comes with an exhaustive set of minimal proofs and logical distractions. Evaluations of state-of-the-art LLMs reveal a clear limitation: models often fixate on one route and increasingly miss alternatives as depth grows. LogicGraph offers a testbed to drive future models toward more flexible, comprehensive reasoning.

8 Acknowledgements

This work was supported by Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (JYB2025XDXM116), National Natural Science Foundation of China (No. 62137002, 62293550, 62293553, 62293554, 62437002, 62477036, 62477037, 62192781), the Shaanxi Provincial Social Science Foundation Project (No. 2024P041), the Youth Innovation Team of Shaanxi Universities "Multi-modal Data Mining and Fusion", and Xi'an Jiaotong University City College Research Project (No. 2024Y01).

Limitations

While LogicGraph represents a significant step towards evaluating complex reasoning, we acknowledge the following limitations:

Synthetic-to-Real Gap. Although we employ advanced LLMs to render symbolic logic into natural language narratives, the dataset remains synthetically constructed. Real-world reasoning often involves ambiguity, fuzzy logic, and probabilistic inference, which are strictly controlled in our current discrete logic framework. The clean logical separation in our benchmark may not fully reflect the noisy information retrieval challenges found in the real world.

Computational Cost. Neuro-symbolic verification of multi-path problems is computationally in-

tensive compared to simple multiple-choice evaluations. This may pose challenges for rapid, low-resource model evaluation.

Ethical Statement

LogicGraph is a fully synthetic benchmark built from abstract entities and explicit logic rules, so it contains no personally identifiable information or copyright-protected text, eliminating risks related to data privacy or intellectual property infringement. A remaining concern is that LLM-based surface realization can introduce latent social biases; we reduce this risk by using constrained templates and distributing instances across multiple neutral domains to avoid skewed coverage. We also acknowledge the environmental cost of multi-path reasoning and neuro-symbolic verification, which can be more computationally intensive than standard evaluations; future work will prioritize more efficient pipelines to reduce the associated carbon footprint. Although stronger reasoning may be repurposed in dual-use settings, the benchmark's abstract, formal scope limits direct harmful applicability. We will release the LogicGraph benchmark artifacts, including dataset instances, generation scripts, and evaluation code, under a permissive open-source license to facilitate reuse and reproducibility.

References

Ajlan Al-Ajlan. 2015. The comparison between forward and backward chaining. *International Journal of Machine Learning and Computing*, 5(2):106.

Anthropic. 2025. Introducing claude sonnet 4.5.

Yifan Bai, Yiping Bao, Y Charles, Cheng Chen, Guanduo Chen, Haiting Chen, Huarong Chen, Jiahao Chen, Ningxin Chen, et al. 2025. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*.

Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. 2024. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 17682–17690.

Bikram Pratim Bhuyan, Amar Ramdane-Cherif, Ravi Tomar, and TP Singh. 2024. Neuro-symbolic artificial intelligence: a survey. *Neural Computing and Applications*, 36(21):12809–12844.

Zhuang Qiang Bok and Watson Wei Khong Chua. 2025. Reasoning beyond the obvious: Evaluating divergent and convergent thinking in llms for financial scenarios. *arXiv preprint arXiv:2507.18368*.

Michael K Chen, Xikun Zhang, and Dacheng Tao. 2025. Justlogic: A comprehensive benchmark for evaluating deductive reasoning in large language models. *arXiv preprint arXiv:2501.14851*.

Peter Clark, Oyvind Tafjord, and Kyle Richardson. 2020. Transformers as soft reasoners over language. *arXiv preprint arXiv:2002.05867*.

Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient smt solver. international conference on tools and algorithms for the construction and analysis of systems, springer.

Leonardo De Moura, Soonho Kong, Jeremy Avigad, Floris Van Doorn, and Jakob von Raumer. 2015. The lean theorem prover (system description). In *International conference on automated deduction*, pages 378–388. Springer.

Google Deepmind. 2025a. Gemini 2.5 flash.

Google Deepmind. 2025b. Gemini 2.5 pro.

Google Deepmind. 2025c. Gemini 3.0 pro.

Google Deepmind. 2026. Gemini 3.5 flash.

DeepSeek-AI. 2025. Introducing deepseek-v3.2-exp.

Shujie Deng, Honghua Dong, and Xujie Si. 2024. Enhancing and evaluating logical reasoning abilities of large language models. In *ICLR 2024 Workshop on Secure and Trustworthy Large Language Models*.

Yuanning Feng, Sinan Wang, Zhengxiang Cheng, Yao Wan, and Dongping Chen. 2025. Are we on the right way to assessing llm-as-a-judge? *arXiv preprint arXiv:2512.16041*.

Joy Paul Guilford. 1967. *The nature of human intelligence.* McGraw-Hill.

Simeng Han, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Wenfei Zhou, James Coady, David Peng, Yujie Qiao, Luke Benson, et al. 2024. Folio: Natural language reasoning with first-order logic. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 22017–22031.

Rishi Hazra, Gabriele Venturato, Pedro Zuidberg Dos Martires, and Luc De Raedt. 2025. Have large language models learned to reason? a characterization via 3-sat phase transition. *arXiv preprint arXiv:2504.03930*.

Kaixuan Huang, Jiacheng Guo, Zihao Li, Xiang Ji, Jiawei Ge, Wenzhe Li, Yingqing Guo, Tianle Cai, Hui Yuan, Runzhe Wang, et al. 2025. Math-perturb: Benchmarking llms' math reasoning abilities against hard perturbations. *arXiv preprint arXiv:2502.06453*.

Robert M. Johnson. 1999. *A Logic Book: Fundamentals of Reasoning*. Wadsworth Publishing Company, Belmont, CA, USA.

Scott Barry Kaufman, Colin G DeYoung, Deidre L Reis, and Jeremy R Gray. 2011. General intelligence predicts reasoning ability even for evolutionarily familiar content. *Intelligence*, 39(5):311–322.

Jian Liu, Leyang Cui, Hanmeng Liu, Dandan Huang, Yile Wang, and Yue Zhang. 2020. Logiqa: A challenge dataset for machine reading comprehension with logical reasoning. *arXiv preprint arXiv:2007.08124*.

William McCune. 2005. Release of prover9. In *Mile high conference on quasigroups, loops and nonassociative systems, Denver, Colorado*.

Meta. 2024. Llama 3.1.

Terufumi Morishita, Gaku Morio, Atsuki Yamaguchi, and Yasuhiro Sogawa. 2023. Learning deductive reasoning from synthetic corpus based on formal logic. In *International Conference on Machine Learning*, pages 25254–25274. PMLR.

OpenAI. 2025a. Gpt-5.1: A smarter, more conversational chatgpt.

OpenAI. 2025b. gpt-oss-120b & gpt-oss-20b model card. *Preprint*, arXiv:2508.10925.

OpenAI. 2025c. Introducing openai o3 and o4-mini.

OpenAI. 2026. Introducing gpt-5.5.

Liangming Pan, Alon Albalak, Xinyi Wang, and William Wang. 2023. Logic-lm: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 3806–3824.

Mihir Parmar, Nisarg Patel, Neeraj Varshney, Mutsumi Nakamura, Man Luo, Santosh Mashetty, Arindam Mitra, and Chitta Baral. 2024. Logicbench: Towards systematic evaluation of logical reasoning ability of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13679–13707.

Nisarg Patel, Mohith Kulkarni, Mihir Parmar, Aashna Budhiraja, Mutsumi Nakamura, Neeraj Varshney, and Chitta Baral. 2024. Multi-logieval: Towards evaluating multi-step logical reasoning ability of large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 20856–20879.

Chengwen Qi, Ren Ma, Bowen Li, He Du, Binyuan Hui, Jinwang Wu, Yuanjun Laili, and Conghui He. 2025. Large language models meet symbolic provers for logical reasoning evaluation. *arXiv preprint arXiv:2502.06563*.

Eleanor Rosch and Carolyn B Mervis. 1975. Family resemblances: Studies in the internal structure of categories. *Cognitive psychology*, 7(4):573–605.

Abulhair Saparov and He He. 2022. Language models are greedy reasoners: A systematic formal analysis of chain-of-thought. *arXiv preprint arXiv:2210.01240*.

Ziju Shen, Naohao Huang, Fanyi Yang, Yutong Wang, Guoxiong Gao, Tianyi Xu, Jiedong Jiang, Wanyi He, Pu Yang, Mengzhou Sun, et al. 2025. Real-prover: Retrieval augmented lean prover for mathematical reasoning. *arXiv preprint arXiv:2505.20613*.

Annalisa Szymanski, Noah Ziems, Heather A Eicher-Miller, Toby Jia-Jun Li, Meng Jiang, and Ronald A Metoyer. 2025. Limitations of the llm-as-a-judge approach for evaluating llm outputs in expert knowledge tasks. In *Proceedings of the 30th International Conference on Intelligent User Interfaces*, pages 952–966.

Oyvind Tafjord, Bhavana Dalvi, and Peter Clark. 2021. Proofwriter: Generating implications, proofs, and abductive statements over natural language. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 3621–3634.

Qwen Team. 2025. Qwq-32b: Embracing the power of reinforcement learning.

Jidong Tian, Yitian Li, Wenqing Chen, Liqiang Xiao, Hao He, and Yaohui Jin. 2021. Diagnosing the first-order logical reasoning ability through logicnli. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3738–3747.

Ruida Wang, Jipeng Zhang, Yizhen Jia, Rui Pan, Shizhe Diao, Renjie Pi, and Tong Zhang. 2024a. Theoreml-lama: Transforming general-purpose llms into lean4 experts. *arXiv preprint arXiv:2407.03203*.

Siyuan Wang, Zhongyu Wei, Yejin Choi, and Xiang Ren. 2024b. Can llms reason with rules? logic scaffolding for stress-testing and improving llms. *arXiv preprint arXiv:2402.11442*.

Yidong Wang, Yunze Song, Tingyuan Zhu, Xuanwang Zhang, Zhuohao Yu, Hao Chen, Chiyu Song, Qiufeng Wang, Cunxiang Wang, Zhen Wu, et al. 2025. Trustjudge: Inconsistencies of llm-as-a-judge and how to alleviate them. *arXiv preprint arXiv:2509.21117*.

Anjiang Wei, Yuheng Wu, Yingjia Wan, Tarun Suresh, Huanmi Tan, Zhanke Zhou, Sanmi Koyejo, Ke Wang, and Alex Aiken. 2025. Satbench: Benchmarking llms' logical reasoning via automated puzzle generation from sat formulas. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 33820–33837.

Fangzhi Xu, Qika Lin, Jiawei Han, Tianzhe Zhao, Jun Liu, and Erik Cambria. 2025. Are large language models really good logical reasoners? a comprehensive evaluation and beyond. *IEEE Transactions on Knowledge and Data Engineering*.

Fangzhi Xu, Zhiyong Wu, Qiushi Sun, Siyu Ren, Fei Yuan, Shuai Yuan, Qika Lin, Yu Qiao, and Jun Liu. 2024a. Symbol-llm: Towards foundational symbol-centric interface for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13091–13116.

Jundong Xu, Hao Fei, Liangming Pan, Qian Liu, Mong-Li Lee, and Wynne Hsu. 2024b. Faithful logical reasoning via symbolic chain-of-thought. *arXiv preprint arXiv:2405.18357*.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, and 22 others. 2024. Qwen2.5 technical report. *CoRR*, abs/2412.15115.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822.

Weihao Yu, Zihang Jiang, Yanfei Dong, and Jiashi Feng. 2020. Reclor: A reading comprehension dataset requiring logical reasoning. *arXiv preprint arXiv:2002.04326*.

Xinyu Zhang, Yuxuan Dong, Yanrui Wu, Jiaxing Huang, Chengyou Jia, Basura Fernando, Mike Zheng Shou, Lingling Zhang, and Jun Liu. 2025. Physreason: A comprehensive benchmark towards physics-based reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16593–16615.

Xinyu Zhang, Yuchen Wan, Boxuan Zhang, Zesheng Yang, Lingling Zhang, Bifan Wei, and Jun Liu. 2026. Dual-cluster memory agent: Resolving multi-paradigm ambiguity in optimization problem solving. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5895–5908.

ZhipuAI. 2025. Glm-4.6.

Wanjun Zhong, Siyuan Wang, Duyu Tang, Zenan Xu, Daya Guo, Jiahai Wang, Jian Yin, Ming Zhou, and Nan Duan. 2021. Ar-lsat: Investigating analytical reasoning of text. *arXiv preprint arXiv:2104.06598*.

A Benchmark

A.1 Argument Forms

In this paper, we adopt the seven most basic argument forms (Johnson, 1999), as summarized in Table 7. Using these core rules, many inference patterns that are not listed explicitly can often be obtained by composing several basic forms. As an illustration, we show how the Destructive Dilemma (DD) can be derived using only the argument forms in Table 7.

Derived rule (DD).

DD: $((p \to q) \wedge (r \to s) \wedge (\neg q \vee \neg s)) \vdash (\neg p \vee \neg r)$.

Derivation using only the forms in Table 7.

1.	$p \to q$	Premise
2.	$r \to s$	Premise
3.	$\neg q \vee \neg s$	Premise
4.	$\neg q \to \neg p$	from 1 by MT
5.	$\neg s \to \neg r$	from 2 by MT
6.	$\neg p \vee \neg r$	from 3, 4, 5 by CD

A.2 Dataset Distribution

Figure 6 visualizes the LogicGraph's distribution along two key dimensions: (1) the average number of reasoning steps (x-axis), which reflects reasoning complexity, and (2) the number of valid ground-truth solutions (y-axis), which reflects solution-space ambiguity. The bubble size and color both indicate the sample frequency at each coordinate.

A notable observation is that the two dimensions are largely independent. The Pearson correlation between average reasoning steps and the number of valid solutions is approximately $\rho \approx 0.06$, indicating a negligible linear relationship. This suggests that our construction effectively decouples reasoning depth from the size of the solution space, enabling controlled evaluation of both complexity and ambiguity within a single benchmark.

A.3 Dataset Domain Diversity Analysis

As illustrated in Table 8, LogicGraph encompasses over 90 distinct specific fields spanning 7 major categories: Business & Finance, Environment & Science, Medical & Health, Law & Governance, Engineering & Tech, Humanities & Society, and Transportation & Urban. The distribution includes high-stakes environments such as Emergency Medicine, Clinical Trials, and Network Security, as well as complex professional scenarios like Regulatory

Argument Form	Formal Notation	Definition (informal)
Modus Ponens (MP)	$((p \to q) \wedge p) \vdash q$	If $p \to q$ and p, then q.
Modus Tollens (MT)	$((p \to q) \wedge \neg q) \vdash \neg p$	If $p \to q$ and $\neg q$, then $\neg p$.
Hypothetical Syllogism (HS)	$((p \to q) \wedge (q \to r)) \vdash (p \to r)$	If $p \to q$ and $q \to r$, then $p \to r$.
Disjunctive Syllogism (DS)	$((p \vee q) \wedge \neg p) \vdash q$	If $p \vee q$ and $\neg p$, then q (symmetrically, $\neg q$ yields p).
Constructive Dilemma (CD)	$((p \to q) \wedge (r \to s) \wedge (p \vee r)) \vdash (q \vee s)$	If $p \to q$, $r \to s$, and $p \vee r$, then $q \vee s$.
Reductio ad Absurdum (RAA)	$((p \to q) \wedge (p \to \neg q)) \vdash \neg p$	If assuming p leads to a contradiction (e.g., both q and $\neg q$), then $\neg p$.
Disjunction Elimination (DE)	$((p \vee q) \wedge (p \to r) \wedge (q \to r)) \vdash r$	If $p \vee q$, and $p \to r$ and $q \to r$, then r.

Table 7: Basic argument forms and their formal notations.

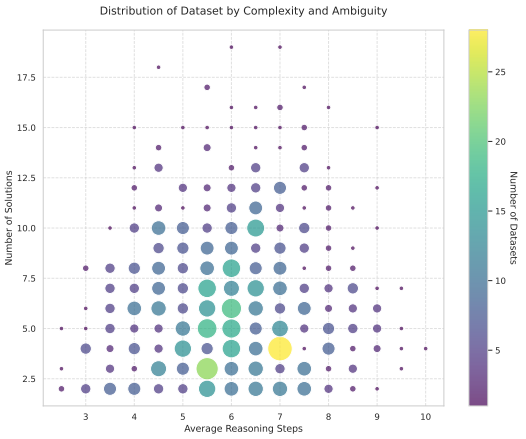


Figure 6: Distribution of the benchmark dataset by average steps and number of solutions. The bubble size indicates the frequency of samples.

Compliance, Industrial Engineering, and Legal Procedures. This diversity ensures that the evaluation reflects the model's ability to handle specialized knowledge and reasoning patterns inherent to different professional and practical domains.

A.4 Concrete Examples

A.4.1 A Concrete DAG Construction Walkthrough

We instantiate the goal-driven procedure of Sec. 4.1 on the access-control example of Figure 1, tracing how the three minimal supports $\{S_{\text{PIN}}, S_{\text{bio}}, S_{\text{escort}}\}$ arise.

Step 1. Goal initialisation. The pipeline samples a target conclusion G : `can_enter(Emma, vault)` as the DAG root.

Step 2. First path (Modus Ponens). To support G, MP introduces parents A_1 : `verified(Emma)` and rule $R_1 : A_1 \to G$ (P6) with fresh identifiers.

Step 3. Recursive expansion. Expanding A_1 via MP introduces A_2 : `pin_ok(Emma)` and rule $R_2 : A_2 \to A_1$ (P4), yielding the PIN-verification path.

Step 4. Branch via node reuse. The intermediate node A_1 is re-selected as a branching point and expanded with fresh atoms A_3 : `fingerprint_match(Emma)` and rule $R_3 : A_3 \to A_1$ (P5), giving the biometric path. The two paths converge at the shared node A_1, producing a DAG rather than a tree.

Step 5. Independent goal branch. The procedure may also branch directly from G: applying MP again introduces A_4 : `escorting(Noah, Emma)` and rule R_4 : $A_4 \to G$ (P7), producing the escort path that shares no intermediate node with the previous two.

Step 6. Semantic instantiation. Once the symbolic DAG is fixed, a coherent domain (here, museum access control) is sampled and an LLM verbalises each atom and rule into the natural-language premises listed in Figure 1.

By construction, the union $\{S_{\text{PIN}}, S_{\text{bio}}, S_{\text{escort}}\}$ is the complete set of minimal supports for G under the given premises, and $|S_{\text{escort}}| = 1$ is the global minimum proof length.

Exhaustiveness by Construction. The paths generated by the backward-chaining procedure (Section 4.1) constitute the complete set of minimal supports for the goal G. This property follows from three structural invariants of the procedure (see table 9).

These invariants jointly imply that (i) the enumerated minimal supports are exhaustive, and (ii) the minimum among their lengths is the *global shortest* proof under the same step accounting used at evaluation time, making the Shortest-Path Finding Rate (Section 5.2) a well-defined metric. Empirically, across >1,500 model outputs on 100 sampled instances, every valid proof had length $\geq$ the constructed minimum, with no counterexamples observed.

Domain Category	Specific Fields (Examples)	# Samples	Percentage
Business & Finance	Quality Control, Insurance, Industrial Engineering, etc.	207	23.0%
Environment & Science	Scientific Method, Biodiversity, Forestry, etc.	187	20.8%
Medical & Health	Clinical Trials, Pharmacology, Emergency Medicine, etc.	167	18.6%
Law & Governance	Regulatory Compliance, Legal Procedure, Litigation, etc.	113	12.6%
Engineering & Tech	IoT, Software Development, Telecommunications, etc.	94	10.4%
Humanities & Society	Sports, Visual Arts, Culinary Arts, etc.	84	9.3%
Other	General Knowledge, Mixed Domains	25	2.8%
Transportation & Urban	Soil Science, Agriculture, Automotive, etc.	23	2.6%

Table 8: Domain Distribution of the LogicGraph Benchmark. The dataset covers a wide spectrum of real-world scenarios, categorized into 7 major domains and over 90 specific fields. This diversity ensures a comprehensive evaluation of reasoning capabilities across various professional and practical contexts.

Invariant	Consequence
Atom uniqueness	Each newly introduced premise receives a globally fresh symbol, so no accidental cross-path entailment can arise.
Disjoint branches	Distinct derivation paths use mutually disjoint premise sets (except at explicitly shared junctions), so premises cannot be "mixed" across paths to form a novel proof.
Topological closure	The DAG is a union of arborescences sharing only G (or explicit reuse nodes), so no "shortcut" through an unshared intermediate exists.

Table 9: Three structural invariants of the backward-chaining DAG construction that jointly guarantee the exhaustiveness of the enumerated minimal supports.

A.4.2 Examples in LogicGraph

To illustrate the three levels of complexity in our dataset, we present one representative example from each category: Small (Figure 7), Medium (Figure 8), and Large (Figure 9). These cases are categorized by the number of distinct reasoning paths available for the given problem.

Test 5 (Small Case) This case represents a scenario with a limited number of paths, containing a total of 4 valid solutions. We provide a detailed look at Solution 2, which features comprehensive annotations including symbolic representations, natural language descriptions, and a complete reasoning chain linking premises to conclusions. Notably, Solution 1 and Solution 4 are classified into the same reasoning family (Family 1). This grouping is due to the presence of shared reasoning nodes between the two paths, demonstrating our method of clustering logically similar derivations.

Test 664 (Medium Case) Representing medium complexity, this test case contains 7 solutions organized into 4 reasoning families. The length of these solutions varies from 5 to 13 steps. The largest family, Family 1, encompasses 4 distinct paths, highlighting the potential for multiple variations within a single line of reasoning.

Test 821 (Hard Case) This case illustrates a high-complexity scenario with 13 solutions distributed across 5 reasoning families. The solution lengths range from 4 to 10 steps. The abundance of alternative paths and families in this example underscores the depth of the logical problems presented.

These examples collectively demonstrate the quality and depth of our dataset. By capturing a wide range of reasoning complexities—from simple, direct derivations to highly branched logical structures—and providing granular annotations, the dataset offers a rigorous standard for evaluating the reasoning capabilities of Large Language Models.

B Error Type Details

B.1 Detailed Definition

Table 10 shows detailed definitions and examples of the error taxonomy.

B.2 Error Distribution

Figure 10 illustrates the distribution of error types (semantic understanding and logical execution) in models, along with a detailed comparison across different error categories.

C Case Study

C.1 Divergent Metrics Reveal Hidden Reasoning Gaps.

We present a representative LogicGraph instance with four ground-truth minimal proof paths orga-

Error Type	Detailed Description	Concrete Example
	A. Semantic Comprehension (Information Processing Layer)	
Semantic Misinterpretation	The model fails to correctly parse the linguistic meaning of a specific fact or rule. This often involves confusing the direction of causality or ignoring negations.	*Context:* "If A, then B." *Model interpretation:* "If B, then A." (Conditional inversion) *Context:* "John is not eligible." *Model interpretation:* "John is eligible." (Negation neglect)
Information Omission	The model completely ignores a critical piece of evidence or a constraint provided in the input context during the reasoning chain.	*Context:* "John has no medical insurance." *Error:* The model concludes John proceeds with a procedure requiring insurance, failing to retrieve the negation fact.
Fact Hallucination	The model fabricates premises or assumes external knowledge that contradicts or is absent from the provided context.	*Context:* "John visits the hospital." *Error:* The model adds "John has cancer" as a premise for deduction, despite no such mention in the text.
	B. Logical Execution (Reasoning Layer)	
Invalid Deduction	The derivation step is logically fundamentally flawed. Even if the premises are correct, the conclusion cannot be derived using standard logic (e.g., formal fallacies).	*Premises:* "Rule: If it rains, the ground is wet." + "Fact: The ground is wet." *Conclusion:* "Therefore, it is raining." *Reason:* Fallacy of affirming the consequent.
Rule Misapplication	The model selects a rule that exists in the context but applies it to an entity or situation where the rule's preconditions are not met.	*Context:* "All doctors must be licensed. John is a driver." *Error:* Applying the "doctor licensing" rule to John, claiming he must have a medical license.
Insufficient Premise	The model draws a definitive conclusion based on a subset of premises that are necessary but not sufficient. It fails to aggregate all required conditions.	*Rule:* "If A and B, then C." *Fact:* "A is true." (B is unknown/false). *Error:* Model concludes "C is true" solely based on A, ignoring the missing condition B.

Table 10: Detailed definitions and examples of the error taxonomy. We provide concrete scenarios to illustrate the distinction between comprehension failures and execution failures.

nized into three distinct reasoning families to qualitatively analyze model behavior beyond aggregate metrics. As shown in Figure 11, all evaluated models achieve perfect convergent performance on this case, with Success Rate, Precision, and Shortest Path Finding Rate all reaching 100%. Under traditional evaluation, these models would therefore appear indistinguishable. This case illustrates that current LLMs tend to conflate reformulation with exploration, highlighting the necessity of explicit divergent evaluation.

However, divergent metrics reveal substantial differences in reasoning behavior. Gemini-3-Pro successfully identifies three out of four valid proof paths, covering all reasoning families (Diversity = 75%, Versatility = 100%). In contrast, o3, despite producing multiple solutions, repeatedly commits to the same dominant derivation, recovering only a single ground-truth path (Diversity = 25%, Versatility = 33%). This indicates early commitment to a high-probability reasoning route, with alternative valid paths remaining unexplored.

A different failure mode is observed for Qwen3-235B-A22B. While the model attempts to enumerate multiple solutions, only two out of six generated paths are logically valid, resulting in a sharp drop in precision. Step-level verification (C.2) shows that invalid solutions often rely on fabricated or insufficient intermediate steps, reflecting result-oriented reasoning rather than structured exploration of the proof space. This highlights that generating more solutions does not necessarily correspond to improved divergent reasoning.

Notably, the reasoning-oriented variant Qwen3-235B-A22B-Thinking achieves full coverage of all ground-truth paths with perfect precision, demon-

strating that explicit reasoning optimization can substantially improve both exploration breadth and logical faithfulness in small multi-path settings. Nevertheless, as shown in our large-scale results, such improvements do not fully eliminate the degradation of diversity under increased reasoning depth.

Overall, this case study illustrates that convergent correctness can mask fundamentally different reasoning strategies. Models may reach the correct conclusion while exhibiting pseudo-divergence—superficially diverse outputs that collapse to the same minimal support—or hallucinated exploration driven by result-oriented fabrication. These behaviors underscore the necessity of explicit divergent evaluation for diagnosing genuine multi-path reasoning ability.

C.2 Diagnosing Logical Fallacies

Extending this analysis to Qwen3-235b-A22b, we observe that high success rates can mask subtle logical flaws in Figure 12. While the models arrive at the correct final answer, a granular inspection of the reasoning chains reveals specific failure modes. Notably, our qualitative analysis identifies that the Thinking model frequently succumbs to *insufficient_premise* errors—making unjustified logical leaps without establishing necessary intermediate lemmas. These findings underscore the necessity of our multi-dimensional evaluation framework for detecting result-oriented fabrications in proof generation.

C.3 Why Formal Verification Is Necessary for Evaluating Logical Reasoning

To illustrate the limitations of current LLM-as-a-judge paradigms and to motivate our formal-verification-based evaluation, we provide a qualitative analysis of three representative scenarios observed in the `test_11` benchmark. These cases expose both (i) false negatives (rejecting valid but compressed reasoning) and (ii) false positives (accepting formally invalid steps) that frequently occur under direct LLM-based evaluation.

C.4 False Negatives on Compressed Logical Inference

LLM-based evaluators often struggle when multiple logical operations are compressed into a single step, which can lead to false negatives. Figure 13 shows a solution produced by qwen3-235b-a22b-thinking. In Step 2, the model effectively applies a CD form, and the premises correspond to:

$$\text{(Rule 2) } \mathsf{ResearchTeam} \vee \mathsf{MedicinalPlant}$$
$$\text{(Rule 1) } \mathsf{ResearchTeam} \rightarrow \neg\mathsf{Infected}$$
$$\text{(Step 1) } \mathsf{MedicinalPlant} \rightarrow \neg\mathsf{Infected}$$

Our verification pipeline (`DeepSeek-V3.2-Exp` with Prover9) successfully proves the target and validates Step 2. However, when using `DeepSeek-V3.2-Exp` directly as an LLM judge (i.e., without formal proof checking), the same step is incorrectly flagged as invalid. This indicates that standard LLM evaluators may lack the robustness to assess logically correct but highly compressed inferences, inadvertently penalizing models that employ higher-efficiency reasoning patterns.

C.5 False Positives and the Gap-Filling Bias of LLM Judges

A more severe failure mode of LLM-as-a-judge is the tendency to over-credit incomplete derivations by implicitly supplying missing premises (i.e., "filling in the gaps"). We compare two `test_11` solutions to demonstrate this issue.

Figure 14(a) presents a correct solution from `qwen3-235b-a22b-thinking`. In Step 2, the derivation explicitly cites the necessary bridge rule:

$$\text{(Rule 5)} \quad \mathsf{Operational} \wedge \mathsf{Dispensers} \rightarrow \mathsf{Controlled},$$

together with the required factual premises. All evaluation methods correctly identify this solution as valid.

Figure 14(b) shows a flawed solution from `qwq-32b`. In Step 2, the model attempts to derive the same conclusion using only:

$$\text{(Step 1) } \mathsf{Operational} \qquad \text{(Fact 2) } \mathsf{Dispensers},$$

while omitting Rule 5. Formally, without the implication rule we have

$$\mathsf{Operational} \wedge \mathsf{Dispensers} \nvdash \mathsf{Controlled},$$

so the step is a non sequitur.

Our formal verifier correctly rejects this step (`False`). In contrast, the other LLM-based evaluators mark it as valid (`True`). This highlights a systematic bias: LLM judges tend to prioritize semantic plausibility over logical entailment, and may implicitly reconstruct missing rules from their own priors or context. As a result, conventional LLM-only evaluation can inflate scores for "nearly correct" but formally invalid proofs, making it unreliable for measuring strict logical reasoning ability.

Model	Convergent (↑) SR Avg; S/M/L	Convergent (↑) Prec. Avg; S/M/L	Convergent (↑) SPF Avg; S/M/L	Divergent (↑) Div. Avg; S/M/L	Divergent (↑) Vers. Avg; S/M/L	Divergent (↑) Orig. Avg; S/M/L	Token Eff. (Overall)↓
General Models							
Llama-3.1-8B-Instruct	**0.33** 0.67/0.33/0.00	**0.13** 0.28/0.12/0.00	**0.22** 0.67/0.00/0.00	**0.11** 0.28/0.06/0.00	**0.19** 0.50/0.07/0.00	**3.71** 5.56/5.56/0.00	N/A
GPT-5.1	**21.56** 19.67/20.67/24.33	**9.69** 10.94/8.76/9.37	**13.56** 14.00/13.67/13.00	**5.98** 8.86/5.02/4.07	**9.96** 13.11/8.39/8.39	**11.51** 8.39/11.65/14.48	7096.54
Qwen2.5-7B-Instruct	**3.80** 11.06/0.00/0.33	**0.25** 0.39/0.00/0.37	**0.15** 0.12/0.00/0.33	**0.23** 0.67/0.00/0.03	**0.25** 0.67/0.00/0.08	**0.34** 1.00/0.00/0.02	N/A
Qwen3-8B	**15.67** 17.67/15.00/14.33	**8.64** 9.30/8.93/7.70	**9.89** 12.67/9.67/7.33	**4.17** 7.69/3.14/1.67	**7.44** 12.56/5.88/3.88	**1.75** 1.69/1.67/1.88	47480.29
Qwen3-14B	**11.34** 15.67/7.67/10.67	**5.87** 7.02/4.72/5.88	**6.89** 10.00/5.00/5.67	**2.93** 5.97/1.61/1.20	**5.25** 9.75/3.22/2.78	**9.14** 8.00/8.82/10.61	58983.61
Reasoning-Oriented Models							
Gemini-2.5-Flash	**85.56** 82.00/85.00/89.67	**67.04** 63.82/67.16/70.14	**65.89** 67.00/66.33/64.33	**41.35** 53.28/40.43/30.33	**61.43** 69.61/60.44/54.24	**17.61** 14.87/16.88/21.07	6835.60

Table 11: Comprehensive model comparison on the LogicGraph, covering convergent metrics (Success Rate/Precision/Short Path Finding Rate), divergent metrics (Diversity/Versatility/Originality), and overall TokenEfficiency (↓) across Small/Medium/Large. In each cell, **Avg** (over Small/Medium/Large) is bolded; the second line reports Small/Medium/Large in order. -T* denotes Thinking.

D Supplementary Experiment

D.1 Main Result

Table 11 lists models that we evaluate but exclude from the main table for space, falling into two categories: (i) weak baselines (Llama-3.1-8B-Instruct, Qwen2.5-7B-Instruct) that score near zero across all metrics and merely confirm that LogicGraph is non-trivial; (ii) family-dominated variants whose stronger sibling already appears in the main table, including the earlier-generation Gemini-2.5-Flash and GPT-5.1.

D.2 Additional Evaluator Robustness

External Human Audit. Three non-author master's students with training in logic independently annotated the same stratified subset of 100 proofs and 400 constituent steps. The proof content and necessary context were shown without model identities, evaluator outputs, or hidden keys. Table 12 shows high pre-adjudication agreement at both levels, indicating that the external judgments used in Table 3 are reproducible across annotators.

Translator-Backbone Robustness. Table 13 reports Diversity after re-scoring fixed outputs from 19 evaluated models on 20 shared instances with DeepSeek-V3.2-Exp, Gemini-2.5-Flash, and GPT-5.5 as the natural-language-to-Prover9 translation backbones. Model outputs and the Prover9 verifier are held fixed. Although absolute scores vary, model rankings remain stable, and no translator materially favors its own model family.

D.3 Controlled Depth Analysis

We isolate depth using three complementary controls: partial correlation, model-fixed-effects regression, and path-count matching. All three analyses retain a negative depth effect after accounting for solution-space and input complexity.

Partial Correlation. Controlling jointly for path, premise, and family counts attenuates the raw correlations, but the association remains negative and significant for every model in Table 14. Thus, structural complexity explains part, but not all, of the observed depth effect.

Fixed-Effects Regression. The pooled analysis in Table 15 confirms that depth independently degrades both correctness and solution coverage. The similar declines in Diversity and Versatility indicate that deeper reasoning restricts exploration at both the path and family levels.

Path-Count Matching. Table 16 compares shallow and deep instances within the same path-count tier. Diversity decreases in every model–tier pair; the near-invariance of Gemini-3.5-Flash on Small instances gives way to a clear gap on Medium and Large instances, reinforcing that depth imposes a burden beyond the size of the solution space.

D.4 ToT / GoT Experiment

Setup. We evaluate ToT (b=3, d=3) and GoT (k=3) on a 150-instance stratified subset (50 per tier: Low / Mid / High), holding the wrapper con-

Level	Shared items	Judgments	Majority (V/I)	Full agreement	Fleiss' κ
Step	400	1,200	203/197	94.50 (378/400)	0.927
Proof	100	300	52/48	88.00 (88/100)	0.840

Table 12: Pre-adjudication inter-annotator agreement among three external annotators.

Model	DeepSeek-V3.2-Exp	Gemini-2.5-Flash	GPT-5.5
Gemini-3.5-Flash	73.64	78.64	78.64
Gemini-3-Pro-Preview	72.67	66.12	66.71
Qwen3-235B-A22B-T*	69.99	59.32	70.99
GPT-5.5	69.32	64.42	72.60
Gemini-2.5-Flash	66.89	55.21	58.96
Gemini-2.5-Pro	65.02	56.30	60.04
DeepSeek-V3.2-Exp-T*	60.11	54.56	60.93
Kimi-K2-T*	55.80	46.05	59.63
o3	38.92	23.70	33.08
DeepSeek-V3.2-Exp	36.26	29.60	36.26
Claude-Sonnet-4.5-T*	30.76	26.05	31.76
QwQ-32B	29.98	17.38	20.60
o4-mini	26.30	22.13	26.30
Qwen3-235B-A22B	20.38	15.00	17.71
GLM-4.6	19.92	20.75	18.25
Claude-Sonnet-4.5	18.92	20.75	21.58
GPT-5.1	11.21	11.93	11.93
GPT-OSS-120B	10.00	9.17	10.00
Qwen3-14B	9.46	5.83	3.75

Table 13: Diversity (%) under three translation backbones on 20 shared instances. Model outputs and the Prover9 verifier are fixed. Pairwise Spearman correlations are 0.970 (DeepSeek-V3.2-Exp–Gemini-2.5-Flash), 0.963 (DeepSeek-V3.2-Exp–GPT-5.5), and 0.982 (Gemini-2.5-Flash–GPT-5.5).

Model	Raw ρ	Controlled ρ	p
Gemini-3.5-Flash	-0.150	-0.095	4.4×10^{-3}
Gemini-3-Pro-Preview	-0.339	-0.144	1.5×10^{-5}
Gemini-2.5-Pro	-0.418	-0.190	8.6×10^{-9}
DeepSeek-V3.2-Exp-T*	-0.472	-0.144	1.5×10^{-5}
Qwen3-235B-A22B-T*	-0.441	-0.164	7.5×10^{-7}

Table 14: Spearman correlations between Diversity and depth over 900 instances. Controlled correlations condition on the numbers of valid paths, premises, and reasoning families.

figuration fixed across three backbones (Gemini-2.5-Flash, DeepSeek-V3.2-Exp, GPT-OSS-120B). Table 17 reports Success Rate per tier and overall.

ToT consistently hurts. ToT degrades Success Rate by a large margin on every backbone (e.g., $84.6 \rightarrow 27.5$ for Gemini-2.5-Flash; $60.4 \rightarrow 8.1$ for DeepSeek-V3.2). Inspection of the intermediate traces shows two failure modes. First, individual branches lack the full premise context, so locally-scored "best" continuations are often globally invalid. Second, the majority-vote aggregation collapses semantically distinct but valid alternatives into a single canonical chain, suppressing the very multi-path behaviour LogicGraph is designed to measure.

Outcome	β_{Depth}	z	p	Effect
Success	-0.434	-9.06	$< 10^{-15}$	Odds $\times 0.65$
Diversity	-0.057	-12.43	$< 10^{-15}$	-5.7 pp
Versatility	-0.059	-10.30	$< 10^{-15}$	-5.9 pp

Table 15: Controlled regressions over 18,000 model–instance pairs with model fixed effects and instance-clustered standard errors. Effects are per unit increase in depth.

GoT is backbone-dependent. For the weakest backbone (GPT-OSS-120B), GoT roughly triples Success Rate ($11.4 \rightarrow 30.2$), consistent with its design rationale of using graph-structured information flow to compensate for limited single-shot reasoning capacity. For stronger backbones, however, the same decomposition fragments coherent end-to-end derivations and significantly underperforms the baseline ($84.6 \rightarrow 51.7$ on Gemini-2.5-Flash; $60.4 \rightarrow 55.7$ on DeepSeek-V3.2). This indicates that structured-decomposition wrappers are not a universal remedy: they may help when the bottleneck is reasoning capacity, but actively interfere when the bottleneck is divergence (i.e., systematic exploration of an already-reachable solution space), which is the regime LogicGraph targets.

Implication for Logic-LM / SymbCoT. The two NL-to-symbolic pipelines closest to our setting, Logic-LM (Pan et al., 2023) and SymbCoT (Xu et al., 2024b), presuppose an upstream natural-language-to-logic translation, which is exactly the auto-formalisation step our evaluator already executes and verifies with Prover9 (Section 5). Re-applying it as part of the reasoner would therefore only compound translation noise and is unlikely to expose any additional capability of LogicGraph beyond what the neuro-symbolic backbone of our pipeline already measures. We therefore view multi-agent collaboration and divergence-aware sampling, rather than translation-heavy single-trace pipelines, as the more promising direction motivated by these findings.

Model	Path tier	Shallow	n	Deep	n	Δ
Gemini-3.5-Flash	Small	68.9	174	68.5	126	−0.4
	Medium	71.5	163	62.4	137	−9.1
	Large	65.8	146	51.2	154	−14.6
Gemini-3-Pro-Preview	Small	76.5	174	61.0	126	−15.5
	Medium	67.6	163	51.6	137	−16.0
	Large	59.3	146	38.2	154	−21.1
Gemini-2.5-Pro	Small	70.7	174	46.6	126	−24.1
	Medium	61.8	163	42.7	137	−19.1
	Large	48.7	146	30.9	154	−17.8
DeepSeek-V3.2-Exp-T*	Small	65.7	174	37.4	126	−28.3
	Medium	48.0	163	28.8	137	−19.2
	Large	36.1	146	16.8	154	−19.3
Qwen3-235B-A22B-T*	Small	69.1	174	46.1	126	−23.0
	Medium	53.1	163	36.3	137	−16.9
	Large	43.2	146	22.6	154	−20.6

Table 16: Diversity (%) for shallow (≤ 6 steps) and deep (> 6 steps) instances within fixed path-count tiers: Small (2–4), Medium (5–7), and Large (≥ 8).

Backbone	Method	Low	Mid	High	Overall
Gemini-2.5-Flash	Baseline	75.5	88.0	90.0	84.6
	ToT	28.6	26.0	28.0	27.5
	GoT	49.0	50.0	56.0	51.7
DeepSeek-V3.2	Baseline	46.9	62.0	72.0	60.4
	ToT	8.2	6.0	10.0	8.1
	GoT	51.0	54.0	62.0	55.7
GPT-OSS-120B	Baseline	10.2	8.0	16.0	11.4
	ToT	0.0	0.0	2.0	0.7
	GoT	26.5	30.0	34.0	30.2

Table 17: Effect of ToT and GoT on LogicGraph (Success Rate, %). Configurations are held fixed across backbones. Green cells indicate an *improvement* over the backbone's Baseline, Red cells indicate a *degradation*.

E Prompt for Dataset Generation

Figure 15 shows the prompt we used to infer a coherent real-world domain from a set of abstract entities and instantiate logical formulas into Prover9 syntax. Below is the prompt we used to translate the instantiated Prover9 expressions into natural language to form the final dataset.

F Prompt for Model Evaluation

Figure 16 shows the prompt we used to test the Large Language Models (LLMs) on the multi-path reasoning task.

G Prompt for Model Performance Evaluation

We present the condensed prompts used in our evaluation pipeline. Dynamic content injected at runtime is denoted by brackets (e.g., CONTEXT).

Figure 17 shows the prompt we used by the evaluator agent to parse the LLM's raw response into structured JSON for automated verification.

H Details of human annotators

For data qualitative evaluation, we engaged four graduate students (including both PhD and Master's students) from engineering disciplines who are also co-authors of this paper. All annotators possessed strong backgrounds in formal logic and critical reasoning, making them well-qualified for this task. Since the annotators were co-authors actively involved in the research, no formal recruitment process or compensation was required, and they were fully aware of how the data would be used in the study. The annotation process focused solely on logical validity and natural language quality evaluation and did not involve collecting any personal identifying information or expose annotators to any risks. As this research involved co-authors analyzing academic content rather than external human subjects, it was determined to be exempt from formal ethics review board approval. The annotation work was conducted as part of regular academic research activities within our institution. No protected or sensitive demographic information was collected or used in this research.

I Details of Ai Assistants In Research Or Writing

Artificial Intelligence tools were utilized in two distinct capacities during this research:

Dataset Generation. To construct the multi-solution logic benchmark, we utilized LLMs

Parameter	Generation Model	Evaluated LLMs	Evaluator Model
Temperature	1.5	0.7	0.1
Max tokens	8,192	10,000	10,000
Design rationale	Encourage diverse, creative instances	Balance determinism with exploratory capacity	Maximize determinism, reliability and consistency

Table 18: Decoding hyperparameters for the three model roles in our pipeline. Temperatures decrease from data generation (most exploratory) to evaluation (most deterministic), while max-token budgets are sized to accommodate long chain-of-thought outputs from the evaluated and evaluator models.

(specifically DeepSeek-V3 via API) to translate symbolic logic templates into natural language narratives. This process involved converting formal logical rules (e.g., implications, disjunctions) into coherent scenarios across various domains such as cybersecurity, law, and medical diagnosis. The generated text was subsequently verified by human annotators to ensure clarity and fidelity to the underlying logic.

Research and Writing Assistance. We utilized GitHub Copilot (powered by Gemini 3 Pro) as a coding assistant to help develop the evaluation scripts and analyze the experimental data.

J Details of Computational Experiment

Our computational experiments evaluated a diverse set of Large Language Models (LLMs) for logical reasoning, covering both proprietary API-based systems and open-weight releases.

Models Evaluated. We evaluated proprietary models accessed via APIs, including GLM-4.6, GPT-5.1, Claude-Sonnet-4.5, o3/o4-mini, and Gemini-2.5-Flash, Gemini-2.5-Pro, Gemini-3-Pro-Preview. We also evaluated open-weight models, including GPT-OSS-120B, Qwen3-235B-A22B (and its Thinking variant) (Yang et al., 2025), QwQ-32B (Team, 2025), and DeepSeek-V3.2-Exp (and its Thinking variant) (DeepSeek-AI, 2025). For all models, we adopted a unified prompting protocol that encourages the model to produce as many independent and verifiable solution paths as possible under a fixed structured answer template; prompt templates are provided in Appendix F.

J.1 Detailed Experimental Configurations

To further ensure reproducibility, we explicitly list all decoding hyperparameters used in our pipeline. Three model roles are distinguished: the *Generation Model* that produces benchmark instances, the *Evaluated LLMs* that are scored on LogicGraph, and the *Evaluator Model* that judges the outputs of the evaluated LLMs. Each role uses a different temperature so that the sampling behaviour matches its functional goal (creative data synthesis vs. balanced inference vs. deterministic judging). The exact settings are summarized in Table 18.

Deployment and Access. All proprietary models were queried through their official APIs. Among the open-weight models, QwQ-32B, Qwen3-8B and Qwen3-14B were deployed locally for inference; they were tested on a machine equipped with two NVIDIA A100 GPUs and two NVIDIA A800 GPUs. All other models were accessed via APIs.

Evaluation Framework. We implemented an automated evaluation pipeline to assess model outputs. In particular, we performed formal verification using the automated theorem prover `Prover9` to check the logical validity of the generated derivations. Only logically valid steps, as verified by `Prover9`, were counted toward the final evaluation metrics.

"Rule1: If the research team receives government grant funding, then they complete the study on drought-resistant wheat",
"Rule2: The research team receives government grant funding or they receive private grant funding",
"Rule3: Provided that the research team has advanced sensor equipment, then if they complete fieldwork during the spring season, they will complete the study on drought-resistant wheat",
"Rule4: If the research team follows ethical guidelines protocol, then they do not publish preliminary findings",
"Rule5: Under the condition that the research team implements changes to their procedures, they request assistance from consultants",
"Rule6: If the research team receives private grant funding, then they achieve the crop yield milestone",
"Rule7: If the review board approves a project extension, then should the private grant funding not lead to study completion, the research team does not modify their research methodology",
"Rule8: The review board either approves a project extension or denies a project extension",
"Rule9: If the research team submits an appeal against the decision, then whenever private grant funding fails to lead to study completion, they do not modify their research methodology",
"Rule10: The research team completes fieldwork during the spring season or they complete fieldwork during the summer season",
"Rule11: Should the research team complete fieldwork during the summer season, then they complete the study on drought-resistant wheat",
"Rule12: Provided that achieving the crop yield milestone does not result in study completion, the research team implements changes to their procedures",
"Rule13: When private grant funding does not trigger study completion, the research team modifies their research methodology",
"Rule14: Under the condition that denying a project extension does not cause the research team to submit an appeal, they accept the ruling on their case",
"Rule15: Should denying a project extension not result in the research team submitting an appeal, then they do not accept the ruling on their case",
"Rule16: If the research team does not follow ethical guidelines protocol, then they violate ethical guidelines protocol",
"Rule17: The research team publishes preliminary findings if they do not complete the study on drought-resistant wheat"
"Fact1: The research team has advanced sensor equipment",
"Fact2: The research team does not violate ethical guidelines protocol",
"Fact3: The research team does not request assistance from consultants"
"Proof Goal": "The research team completes the study on drought-resistant wheat"

Family 1:

- Solution 1 (2 steps):
  Step 1 (Rule: MP):
  Premises:
  - Rule3: (has_equipment(ResearchTeam, AdvancedSensors) -> (completes_fieldwork(ResearchTeam, SpringSeason) -> completes_study(ResearchTeam, DroughtResistantWheat)))
    Natural: Provided that the research team has advanced sensor equipment, then if they complete fieldwork during the spring season, they will complete the study on drought-resistant wheat
  - Fact1: has_equipment(ResearchTeam, AdvancedSensors)
  Natural: The research team has advanced sensor equipment
  Step1_Conlusion: The research team completes the study on drought-resistant wheat if they complete fieldwork during the spring season
  Step 2 (Rule: DE):
  Premises:
  - Rule10: (completes_fieldwork(ResearchTeam, SpringSeason) | completes_fieldwork(ResearchTeam, SummerSeason))
    Natural: The research team completes fieldwork during the spring season or they complete fieldwork during the summer season
  - Derived: (completes_fieldwork(ResearchTeam, SpringSeason) -> completes_study(ResearchTeam, DroughtResistantWheat))
    Natural: The research team completes the study on drought-resistant wheat if they complete fieldwork during the spring season
  - Rule11: (completes_fieldwork(ResearchTeam, SummerSeason) -> completes_study(ResearchTeam, DroughtResistantWheat))
    Natural: Should the research team complete fieldwork during the summer season, then they complete the study on drought-resistant wheat
    Step2_Conlusion: The research team completes the study on drought-resistant wheat

Family 2:

- Solution 2 (5 steps):
  Step 1: RAA + [Rule14, Rule15] => Derived1
  Step 2: HS + [Derived1, Rule9] => Derived2
  Step 3: DE + [Rule8, Rule7, Derived2] => Derived3
  Step 4: RAA + [Rule13, Derived3] => Derived4
  Step 5: DE + [Rule2, Rule1, Derived4] => *G*
- Solution 3 (4 steps):
  Step 1: HS + [Rule12, Rule5] => Derived1
  Step 2: MT + [Derived1, Fact3] => Derived2
  Step 3: HS + [Rule6, Derived2] => Derived3
  Step 4: DE + [Rule2, Rule1, Derived3] => *G*

Family 3:

- Solution 4 (3 steps): Step 1: MT + [Rule16, Fact2] => Derived1;
  Step 2: MP + [Rule4, Derived1] => Derived2; Step 3: MT + [Rule17, Step2] => *G*

Figure 7: A Small type example in LogicGraph.

"Rule1: Either the EPA has approved research funding for the climate study or the EPA has approved research funding for the biodiversity survey.",
"Rule2: If samples have been processed in the Arctic region, then preliminary data has been analyzed in the Arctic region.",
...
"Rule41: If environmental protection goals have not been met by the EPA, then regulation compliance has been met by the EPA.",
"Rule42: If environmental protection goals have not been met by the EPA, then regular audits have been conducted by the EPA."
"Fact1: The contingency plan has been activated by the EPA.",
...
"Fact11: Project milestones have not been achieved by the EPA."
"Proof Goal": "Environmental protection goals have been met by the EPA."

Family 1:

- Solution 1 (7 steps):
  Step 1: HS + [Rule26, Rule27] => Derived1
  Step 2: MP + [Derived1, Fact4] => Derived2
  Step 3: RAA + [Derived2, Rule35] => Derived3
  Step 4: DE + [Derived3, Rule24, Rule25] => Derived4
  Step 5: MP + [Rule20, Derived4] => Derived5
  Step 6: HS + [Derived5, Rule11] => Derived6
  Step 7: DE + [Rule1, Derived6, Rule5] => *G*
- Solution 5 (5 steps):
  Step 1: MP + [Rule14, Fact1] => Derived1
  Step 2: MP + [Rule13, Derived1] => Derived2
  Step 3: DS + [Rule12, Derived2] => Derived3
  Step 4: MP + [Rule10, Derived3] => Derived4
  Step 5: DE + [Rule1, Derived4, Rule5] => *G*
- Solution 6 (8 steps):
  Step 1: MP + [Rule19, Fact2] => Derived1
  Step 2: DE + [Rule18, Rule17, Derived1] => Derived2
  Step 3: RAA + [Rule15, Derived2] => Derived3
  Step 4: DS + [Rule16, Derived3] => Derived4
  Step 5: RAA + [Rule40, Derived4] => Derived5
  Step 6: MP + [Rule20, Derived5] => Derived6
  Step 7: HS + [Derived6, Rule11] => Derived7
  Step 8: DE + [Rule1, Derived7, Rule5] => *G*
- Solution 7 (13 steps):
  Step 1: MT + [Rule22, Fact11] => Derived1
  Step 2: DS + [Rule23, Derived1] => Derived2
  Step 3: RAA + [Rule38, Derived2] => Derived3
  Step 4: RAA + [Rule34, Derived3] => Derived4
  Step 5: MP + [Derived4, Fact3] => Derived5
  Step 6: DS + [Derived5, Fact10] => Derived6
  Step 7: DS + [Rule21, Derived6] => Derived7
  Step 8: MT + [Derived7, Fact9] => Derived8
  Step 9: RAA + [Derived8, Rule35] => Derived9
  Step 10: DE + [Derived9, Rule24, Rule25] => Derived10
  Step 11: MP + [Rule20, Derived10] => Derived11
  Step 12: HS + [Derived11, Rule11] => Derived12
  Step 13: DE + [Rule1, Derived12, Rule5] => *G*

Family 2:

- Solution 2 (7 steps):
  Step 1: RAA + [Rule36, Rule37] => Derived1
  Step 2: RAA + [Rule30, Derived1] => Derived2
  Step 3: DS + [Derived2, Fact6] => Derived3
  Step 4: HS + [Derived3, Rule2] => Derived4
  Step 5: HS + [Rule28, Derived4] => Derived5
  Step 6: RAA + [Derived5, Rule29] => Derived6
  Step 7: DS + [Derived6, Fact5] => *G*

Family 3:

- Solution 4 (7 steps):
  Step 1: MT + [Rule33, Fact8] => Derived1
  Step 2: HS + [Derived1, Rule9] => Derived2
  Step 3: HS + [Derived2, Rule8] => Derived3
  Step 4: HS + [Rule39, Derived3] => Derived4
  Step 5: MT + [Derived4, Fact7] => Derived5
  Step 6: MP + [Rule7, Derived5] => Derived6
  Step 7: MP + [Rule6, Derived6] => *G*

Family 4:

- Solution 3 (5 steps):
  Step 1: RAA + [Rule31, Rule32] => Derived1
  Step 2: HS + [Rule42, Derived1] => Derived2
  Step 3: HS + [Derived2, Rule4] => Derived3
  Step 4: HS + [Derived3, Rule3] => Derived4
  Step 5: RAA + [Rule41, Derived4] => *G*

Figure 8: A Medium type example in LogicGraph.

"Rule1: Either the stream is live or the tournament is successful.",
"Rule2: Provided that the match needs an update, the schedule is finalized or it is not approved.",
...
"Rule60: If the event is not scheduled, then the event is cancelled.",
"Rule61: Provided that the tournament is not successful, the platform does not have a backup."
"Fact1: The team has transport.",
...
"Fact13: It is not the case that the schedule is finalized."
"Proof Goal": "The tournament is successful."

Family 1:

- Solution 1 (5 steps):
  Step 1: MP + [Rule40, Fact3] => Derived1
  Step 2: MP + [Derived1, Fact2] => Derived2
  Step 3: DE + [Rule21, Rule20, Derived2] => Derived3
  Step 4: DS + [Derived3, Fact5] => Derived4
  Step 5: DS + [Rule1, Derived4] => *G*

Family 2:

- Solution 2 (4 steps):
  Step 1: DE + [Rule3, Rule2, Rule4] => Derived1
  Step 2: DS + [Derived1, Fact13] => Derived2
  Step 3: DS + [Rule44, Derived2] => Derived3
  Step 4: DS + [Rule43, Derived3] => *G*
- Solution 6 (6 steps):
  Step 1: RAA + [Rule48, Rule49] => Derived1
  Step 2: CD + [Rule22, Rule23, Derived1] => Derived2
  Step 3: DS + [Derived2, Fact8] => Derived3
  Step 4: DS + [Derived3, Fact13] => Derived4
  Step 5: DS + [Rule44, Derived4] => Derived5
  Step 6: DS + [Rule43, Derived5] => *G*

Family 3:

- Solution 5 (5 steps):
  Step 1: RAA + [Rule55, Rule56] => Derived1
  Step 2: MT + [Derived1, Fact7] => Derived2
  Step 3: HS + [Rule19, Derived2] => Derived3
  Step 4: HS + [Rule18, Derived3] => Derived4
  Step 5: DE + [Rule17, Rule16, Derived4] => *G*
- Solution 11 (8 steps):
  Step 1: DS + [Rule37, Fact11] => Derived1
  Step 2: HS + [Rule51, Derived1] => Derived2
  Step 3: RAA + [Rule50, Derived2] => Derived3
  Step 4: DE + [Derived3, Rule35, Rule36] => Derived4
  Step 5: RAA + [Rule47, Derived4] => Derived5
  Step 6: HS + [Rule19, Derived5] => Derived6
  Step 7: HS + [Rule18, Derived6] => Derived7
  Step 8: DE + [Rule17, Rule16, Derived7] => *G*
- Solution 12 (6 steps):
  Step 1: RAA + [Rule57, Rule58] => Derived1
  Step 2: RAA + [Derived1, Rule52] => Derived2
  Step 3: RAA + [Rule38, Derived2] => Derived3
  Step 4: DS + [Rule39, Derived3] => Derived4
  Step 5: HS + [Rule18, Derived4] => Derived5
  Step 6: DE + [Rule17, Rule16, Derived5] => *G*
- Solution 13 (4 steps):
  Step 1: DS + [Rule42, Fact12] => Derived1
  Step 2: MP + [Rule41, Derived1] => Derived2
  Step 3: HS + [Rule18, Derived2] => Derived3
  Step 4: DE + [Rule17, Rule16, Derived3] => *G*

Family 4:

- Solution 4 (6 steps):
  Step 1: MP + [Rule15, Fact1] => Derived1
  Step 2: DS + [Rule14, Derived1] => Derived2
  Step 3: MT + [Rule45, Derived2] => Derived3
  Step 4: DS + [Derived3, Fact6] => Derived4
  Step 5: RAA + [Rule59, Derived4] => Derived5
  Step 6: RAA + [Derived5, Rule61] => *G*
- Solution 8 (4 steps):
  Step 1: DE + [Rule27, Rule26, Rule28] => Derived1
  Step 2: DS + [Rule25, Derived1] => Derived2
  Step 3: RAA + [Rule59, Derived2] => Derived3
  Step 4: RAA + [Derived3, Rule61] => *G*
- Solution 9 (10 steps):
  Step 1: RAA + [Rule53, Rule54] => Derived1
  Step 2: HS + [Derived1, Rule30] => Derived2
  Step 3: RAA + [Rule46, Derived2] => Derived3
  Step 4: HS + [Derived3, Rule29] => Derived4
  Step 5: RAA + [Derived4, Rule13] => Derived5
  Step 6: DS + [Rule14, Derived5] => Derived6
  Step 7: MT + [Rule45, Derived6] => Derived7
  Step 8: DS + [Derived7, Fact6] => Derived8
  Step 9: RAA + [Rule59, Derived8] => Derived9
  Step 10: RAA + [Derived9, Rule61] => *G*
- Solution 10 (9 steps):
  Step 1: CD + [Rule32, Rule34, Rule33] => Derived1
  Step 2: DS + [Derived1, Fact10] => Derived2
  Step 3: DS + [Rule31, Derived2] => Derived3
  Step 4: MT + [Rule12, Derived3] => Derived4
  Step 5: DS + [Rule14, Derived4] => Derived5
  Step 6: MT + [Rule45, Derived5] => Derived6
  Step 7: DS + [Derived6, Fact6] => Derived7
  Step 8: RAA + [Rule59, Derived7] => Derived8
  Step 9: RAA + [Derived8, Rule61] => *G*

Family 5:

- Solution 3 (7 steps):
  Step 1: MT + [Rule10, Fact4] => Derived1
  Step 2: MT + [Rule9, Derived1] => Derived2
  Step 3: MT + [Rule7, Derived2] => Derived3
  Step 4: DS + [Rule8, Derived3] => Derived4
  Step 5: RAA + [Rule6, Derived4] => Derived5
  Step 6: MT + [Rule60, Derived5] => Derived6
  Step 7: MP + [Rule5, Derived6] => *G*
- Solution 7 (8 steps):
  Step 1: HS + [Rule11, Rule24] => Derived1
  Step 2: MT + [Derived1, Fact9] => Derived2
  Step 3: MT + [Rule9, Derived2] => Derived3
  Step 4: MT + [Rule7, Derived3] => Derived4
  Step 5: DS + [Rule8, Derived4] => Derived5
  Step 6: RAA + [Rule6, Derived5] => Derived6
  Step 7: MT + [Rule60, Derived6] => Derived7
  Step 8: MP + [Rule5, Derived7] => *G*

Figure 9: A Large type example in LogicGraph.

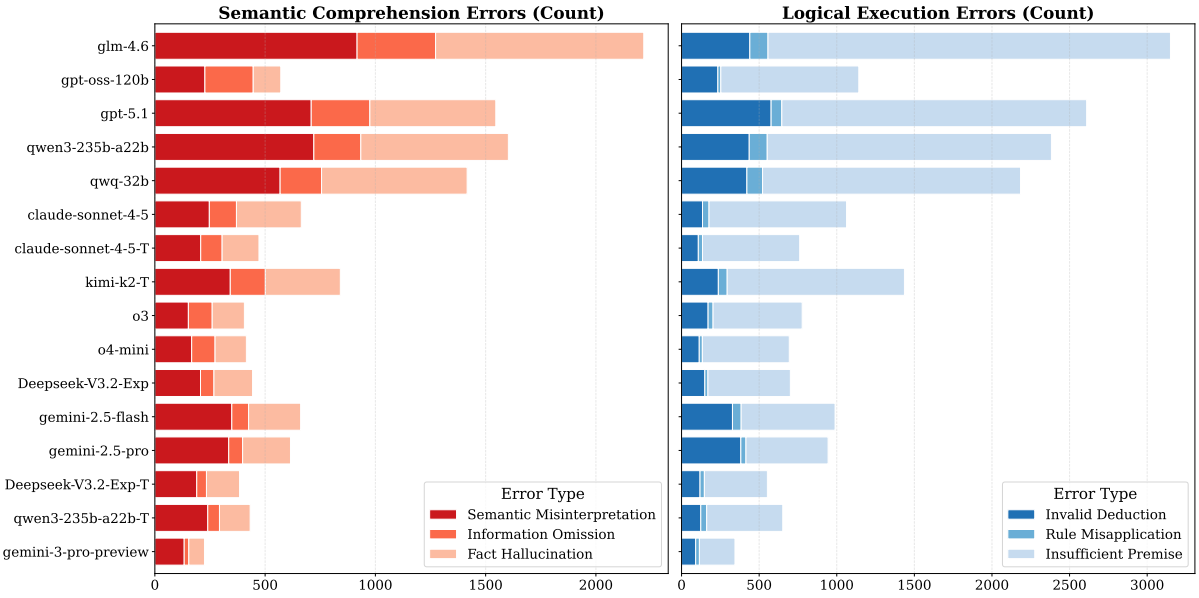


Figure 10: Error distribution across models, grouped by semantic comprehension and logical execution.

	Covergent Metric	Divergent Metric
• Gemini-3-Pro solution1 ✓ (GT4) solution2 ✓ (GT1) solution3 ✓ (GT3)	Success Rate: 100% Precision: 100% SPF: 100%	Diversity: 3/4=75% Versatility: 3/3=100% Originality: 1/2+1/4+1/3=1.58
• o3 solution1 ✓ (GT1)	Success Rate: 100% Precision: 100% SPF: 100%	Diversity: 1/4=25% Versatility: 1/3=33.3% Originality: 1/4=0.25
• Qwen3-235b solution1 ✓ (GT1) solution2 ✓ (GT3) solution3~6 ✗	Success Rate: 100% Precision: 2/6=33.3% SPF: 100%	Diversity: 1/4=25% Versatility: 2/3=66.7% Originality: 1/4+1/3=0.58
• Qwen3-235b-T* solution1 ✓ (GT4) solution2 ✓ (GT1) solution3 ✓ (GT3) solution4 ✓ (GT2)	Success Rate: 100% Precision: 100% SPF: 100%	Diversity: 4/4=100% Versatility: 3/3=100% Originality: 1/2+1/4+1/3+1=2.08

Figure 11: Model performance on case (Figure 7). GT represents the ground truth paths.

Model Response:
Solution 3:
Step 1: [Rule10] => The research team completes fieldwork during the spring season or they complete fieldwork during the summer season (Used: [Rule10])
Step 2: [Rule11, Step1] => The research team completes the study on drought-resistant wheat (Used: [Rule11, Step1])

Evaluation Result: Step 1 ✓ Step 2 ✗ Proof ✗

Error Type: insufficient_premise.

Error Analysis:
The reasoning step attempts to deduce 'The research team completes the study on drought-resistant wheat' using Rule11 and Step1. However, the premises cited are insufficient to support this conclusion. Rule11 (in Prover9) states: 'completes_fieldwork(ResearchTeam, SummerSeason) -> completes_study(ResearchTeam, DroughtResistantWheat)'. To apply this rule, we need the antecedent 'completes_fieldwork(ResearchTeam, SummerSeason)' to be true. Step1 is not provided in the context, but even if Step1 established some fact, it does not appear to provide 'completes_fieldwork(ResearchTeam, SummerSeason)'. Without this necessary premise, the conclusion cannot be logically derived from the cited rule alone.

Figure 12: A specific example of our model's error analysis.

Test 11 qwen3-235b-a22b-thinking-2507 solution2:
Step 1: Fact1, Rule3 => If the medicinal plant is blooming during spring season, then the deer population is not infected with Chronic Wasting Disease (Used: Fact1, Rule3)
Step 2: Rule2, Rule1, Step1 => The deer population is not infected with Chronic Wasting Disease (Used: Rule2, Rule1, Step1)
Step 3: Step2, Rule6 => Chronic Wasting Disease is controlled in the deer population (Used: Step2, Rule6)

	Step 1	Step 2	Proof	Proof
Our (Deepseek-V3.2-Exp)	True ✓	True ✓	True ✓	True ✓
Deepseek-V3.2-Thinking	True ✓	True ✓	True ✓	True ✓
Gemini-3-Pro	True ✓	True ✓	True ✓	True ✓
Deepseek-V3.2-Exp	True ✓	False ✗	False ✗	False ✗

Figure 13: A valid complex reasoning step correctly verified by our formal method but incorrectly rejected by an LLM-based judge (using DeepSeek-V3.2-Exp directly as an LLM judge).

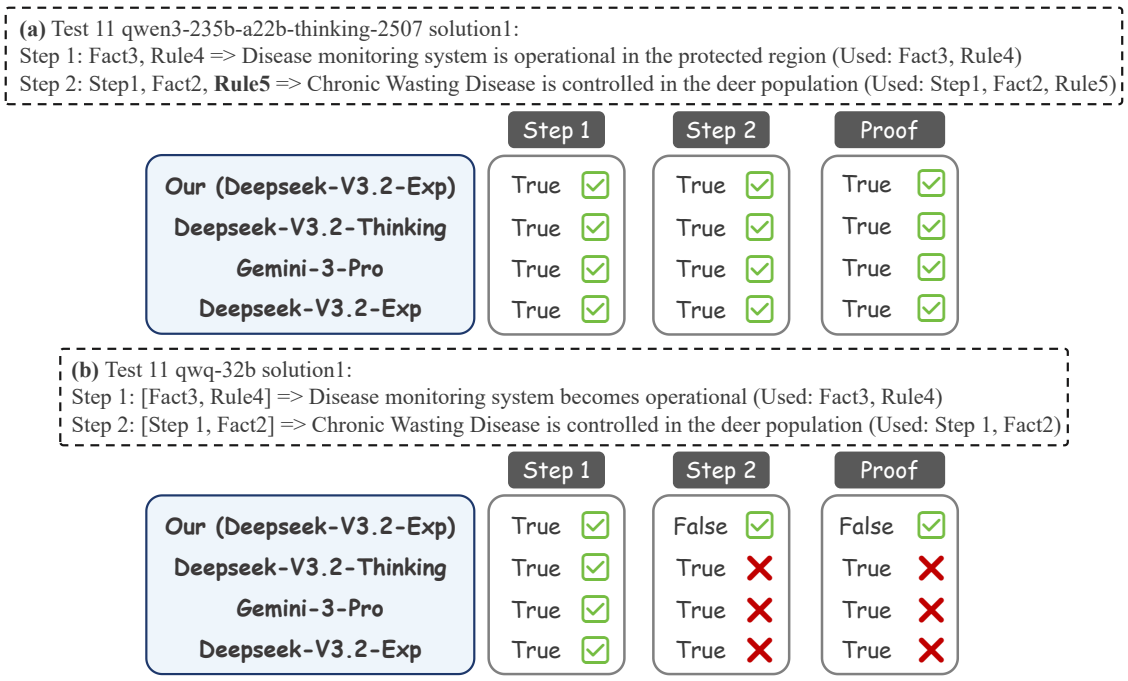


Figure 14: Comparison of evaluation robustness. (a) A fully valid solution. (b) An invalid solution with missing premises (Rule 5), correctly rejected by our method but incorrectly accepted by LLM judges.

System Role: You are a creative assistant. Your task is to infer a coherent real-world domain and instantiate the provided logical formulas strictly in Prover9 syntax.
User Prompt:
Step 1: Infer a Domain
Consider this list of abstract seed entities:
{"Person", "Animal", "Plant", "Food", "Alcohol", "Disease", "Drug", "Natural Phenomenon", "Condition", "Material", "Substance", "Furniture", "Publication", "Organization", "Authorization", "Facility", "Natural Place", "Event", "Show", "Artwork", "Job", "Game", "Vehicle", "Tool", "Technology", "Electronic Device", "Platform", "Financial Product", "Skill", "Legislation", "Region", "Time Period"}.
Infer a single real-world scenario that connects them (e.g., Scientific Research, Medieval Politics, Pharmaceutical Manufacturing).
Step 2: Instantiate Logical Nodes (Prover9 only)
Using the inferred domain, instantiate each logical node below. All entities and relations must belong to the SAME coherent story. Do not produce natural language descriptions in this step.
{{logical_nodes_list}}
Instructions:
1. Atomic Fact Definition (Prover9 only)
  - For every atomic symbol (A1, A2, ...), define a fact using `predicate(entity1, entity2, ...)`.
  - Keep entity names consistent across all expressions.
2. Expression Instantiation (Prover9 only)
  - For each formula provided, instantiate it using the atomic facts you created.
  - Preserve the original logical structure exactly (operators &, |, ->, -, <->).
3. Strict Output Format (Markdown)
Domain Theme: [Single coherent domain]

Entity Instantiation & Atomic Facts (Prover9)
A1: predicate(entity1, entity2, ...)
...
Expression Instantiation (Prover9)
Expression 1: (...)
...

System Role: You are a precise translator. Your job is to convert Prover9 expressions into natural language without losing any logical information.
User Prompt:
Domain Theme: {{domain_theme}}
Atomic Facts (Prover9 syntax): {{atomic_facts}}
Composite Expressions (Prover9 syntax, maintain order): {{composite_expressions}}

Translation Requirements:
- Translate every atomic fact into a clear sentence.
- Translate every composite expression in the SAME order as provided.
- Preserve all entities, logical operators, and conditional structures exactly.
- For implications, alternate conditional phrasing styles (e.g., 'If A, then B', 'Provided that A, B occurs', 'Whenever A happens, B follows', 'Under the condition that A, B follows). Do not reuse the exact same template for multiple implications.
- For conjunctions, mention all conjuncts (e.g., 'Both A and B').
- For disjunctions, vary the connective wording (e.g., 'Either A or B', 'A happens or B happens').
- For negations, state the negation clearly (e.g., 'It is not the case that A', 'A does not hold').
- Do not add or remove entities.
- Use varied but precise phrasing while preserving meaning.

Strict Output Format (Markdown):
Atomic Facts (Natural Language)
A1: sentence
...
Expression (Natural Language)
Expression 1: sentence
...

Figure 15: Prompt we use during the generation process.

Given the following Facts and Rules as the assumptions and conditions of a logical scenario, your task is to find **all** distinct and valid proof paths that derive the conclusion '{question}'.

FACTS:
{facts}
RULES:
{rules}

IMPORTANT NOTATION:
1. Do NOT introduce external knowledge.
2. If NO valid proof path exists, state: "No valid proof path exists for the conclusion '{question}' based on the given facts and rules.
3. Follow the following Strict Output Format:
Solution [number]:
Step 1: [Fact x, Rule y] => [intermediate_conclusion] (Used: [Fact x, Rule y])
Step 2: [Step 1, Fact z] => [Next_intermediate_conclusion] (Used: [Step 1, Fact z])
...
Step n: [Step m, Fact w] => [Final deduction] (Used: [Step m, Fact w])
...

Figure 16: Prompt we use during the test process.

(a) Solution Extraction Prompt

Task: Extract solution paths from the LLM response while preserving ALL original details and structure.
Input:
- Knowledge Base: {{RULES_AND_FACTS}}
- Response: {{LLM_RESPONSE}}

Critical Constraints:
- Preserve all original text, reasoning details, and case-by-case analysis.
- Do not simplify or summarize; keep original step numbering.
- Normalize references (e.g., expand "Steps 1-3" to "Step 1, Step 2, Step 3").

Output Format: JSON object containing a list of solutions, where each solution includes:
- id: Unique identifier.
- steps: List of strings, each containing the full original text of a step.
- conclusion: The final conclusion statement.

(b) Natural Natural Language to Symbolic Expression Conversion Prompt

Task: Convert the given logical reasoning step into valid Prover9 format.
Input:
- Step Text: {{STEP_TEXT}}
- Available Premises: {{PREMISES_LIST}} (Facts, Rules, and verified conclusions from previous steps)
- Context: {{TRANSLATION_MAPPINGS}}

Instructions:
- Identify premises explicitly used in the step text.
- Assumptions Block: Include ONLY the needed premises from the Available Premises list.
- Constraint: When referencing previous steps, use the EXACT Prover9 expression provided, without modification.
- Goals Block: Include only the conclusion derived in this specific step.
- Special Handling: For proof by contradiction, place the positive conclusion in the goals section and do not include the negated assumption in assumptions.

Output: A valid Prover9 block containing formulas(assumptions). ... end_of_list. and formulas(goals). ... end_of_list.

(c) Error Analysis Prompt

Task: Classify the logical error in a failed step using two independent dimensions.
Input:
Context: {{STEP_TEXT}}, {{KNOWLEDGE_BASE}}, {{PROVER9_INPUT/OUTPUT}}
Classification Framework:
Dimension A: Input Processing (Optional)
A1. Semantic Misinterpretation: Model misunderstood the meaning of facts/rules (e.g., reversed conditions).
A2. Information Omission: Model ignored critical given facts/rules.
A3. Fact Hallucination: Model used facts not provided in the input.
Dimension B: Reasoning Process (Mandatory)
B1. Invalid Deduction: Logically invalid inference (e.g., affirming the consequent).
B2. Rule Misapplication: Rule exists but conditions are not satisfied.
B3. Insufficient Premise: Cited premises are incomplete to support the conclusion.
Output Format: JSON object containing:
dimension_a: { "has_error": bool, "error_type": "A1"|"A2"|"A3"|null }
dimension_b: { "error_type": "B1"|"B2"|"B3" }
evidence: Specific quotes or reasoning supporting the classification.

Figure 17: Prompt template we use during the performance evaluation process.